

BECE102L-DIGITAL SYSTEMS DESIGN

MOD-3 DESIGN OF COMBINATIONAL LOGIC CIRCUITS

by

Dr. Penchalaiah Palla

Department of Micro and Nanoelectronics

School of Electronics Engineering

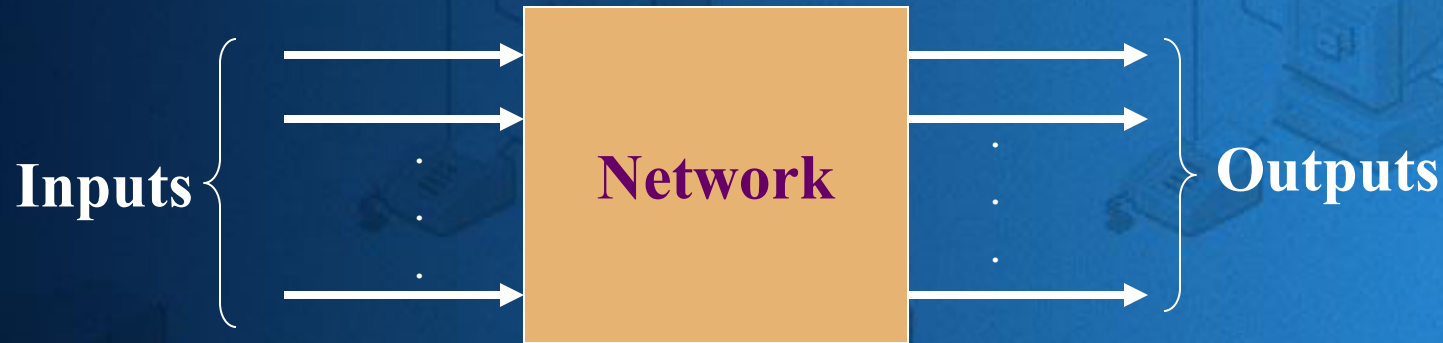
Vellore Institute of Technology

Vellore, TN

BECE102L	Digital Systems Design	L	T	P	C
		3	0	0	3
Pre-requisite	Nil	Syllabus version			
		1.0			
Course Objectives					
1. Provide an understanding of Boolean algebra and logic functions. 2. Develop the knowledge of combinational and sequential logic circuit design. 3. Design and model the data path circuits for digital systems. 4. Establish a strong understanding of programmable logic. 5. Enable the student to design and model the logic circuits using Verilog HDL.					
Course Outcome					
At the end of the course the student will be able to 1. Optimize the logic functions using and Boolean principles and K-map 2. Model the Combinational and Sequential logic circuits using Verilog HDL 3. Design the various combinational logic circuits and data path circuits 4. Analyze and apply the design aspects of sequential logic circuits 5. Analyze and apply the design aspects of Finite state machines 6. Examine the basic architectures of programmable logic devices					

Module:1	Digital Logic	8 hours
Boolean Algebra: Basic definitions, Axiomatic definition of Boolean Algebra, Basic Theorems and Properties of Boolean Algebra, Boolean Functions, Canonical and Standard Forms, Simplification of Boolean functions. Gate-Level Minimization: The Map Method (K-map up to 4 variable), Product of Sums and Sum of Products Simplification, NAND and NOR Implementation. Logic Families: Digital Logic Gates, TTL and CMOS logic families.		
Module:2	Verilog HDL	5 hours
Lexical Conventions, Ports and Modules, Operators, Dataflow Modelling, Gate Level Modelling, Behavioural Modeling, Test Bench		
Module:3	Design of Combinational Logic Circuits	8 hours
Design Procedure, Half Adder, Full Adder, Half Subtractor, Full Subtractor, Decoders, Encoders, Multiplexers, De-multiplexers, Parity generator and checker, Applications of Decoder, Multiplexer and De-multiplexer. Modeling of Combinational logic circuits using Verilog HDL.		
Module:4	Design of data path circuits	6 hours
N-bit Parallel Adder/Subtractor, Carry Look Ahead Adder, Unsigned Array Multiplier, Booth Multiplier, 4-Bit Magnitude comparator. Modeling of data path circuits using Verilog HDL		

Remember



◆ Combinational

- The outputs depend only on the current input values
- It uses only logic gates

◆ Sequential

- The outputs depend on the current and past input values
- It uses logic gates and storage elements

Notes

- ◆ If there are n input variables, there are 2^n input combinations
- ◆ For each input combination, there is one output value
- ◆ Truth tables are used to list all possible combinations of inputs and corresponding output values

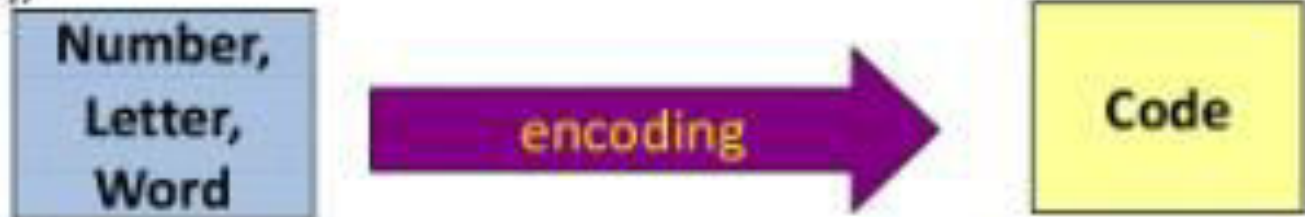
Basic Combinational Circuits

- ◆ Code Converters
- ◆ Adders
- ◆ Subtractors
- ◆ Decoders
- ◆ Encoders
- ◆ Multiplexers
- ◆ Demultiplexers

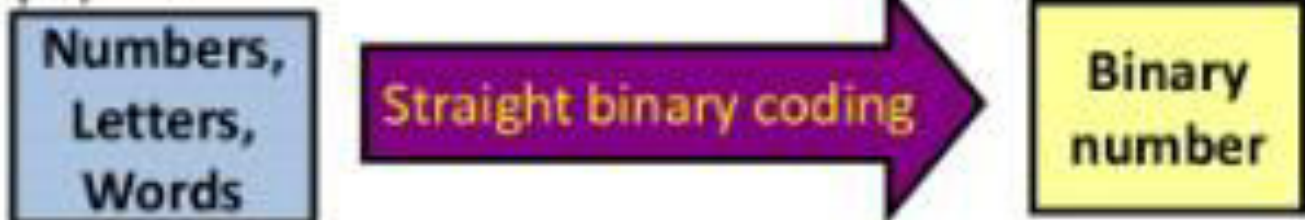
Code and Encoding

- Code-- Special group of symbols that is used to represent numbers, letters or words.
- Encoding-- The process of converting a number/letter/word into a code.

Generally,



For example,



Other Codes

- External world is decimal but digital systems use binary numbers.
 - So, conversions are often.
- Other codes are introduced because the conversion between the code and the decimal number is easier.

BCD Code

- **B**inary **C**oded **D**ecimal Code
- Binary Coded Decimal (BCD)—a way to represent each digit of a decimal number with its 4-bit binary number.

Example: 874_{10}

Decimal	8	7	4
	↓	↓	↓
BCD	1000	0111	0100

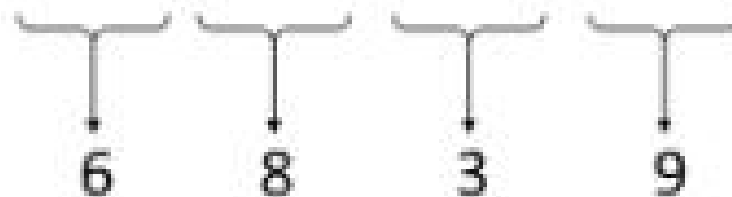
Do you know why?

1000 1111 is not a BCD

Therefore, the BCD code for 874_{10} is 1000 0111 0100

BCD Code / 8421 Code

- Convert a BCD code to its decimal equivalent.
 - Step 1: Break the BCD into 4-bit groups, starting from LSB
 - Step 2: Replace each 4-bit group with its equivalent decimal
- Example: 0110 1000 0011 1001



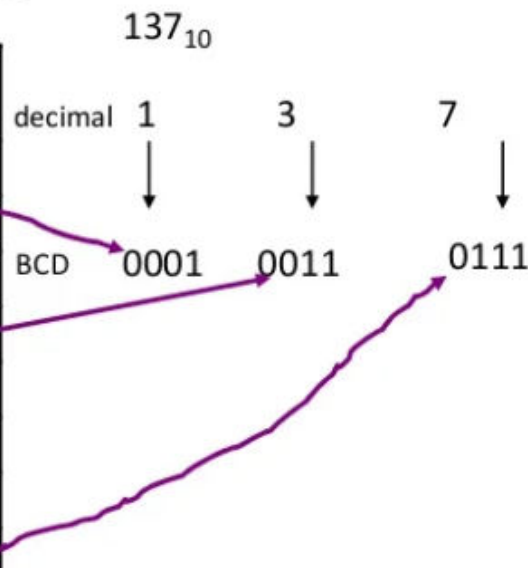
So, the decimal equivalent is 6839_{10}

BCD coding vs. straight binary coding

- BCD coding is easier.

BCD coding

Decimal	Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001



The BCD code is 0001 0011 0111

$$\begin{array}{rcl}
 137_{10} & & \\
 \hline
 137 & =68 & 1 \\
 2 & & \uparrow \\
 \hline
 68 & =34 & 0 \\
 2 & & \\
 \hline
 34 & =17 & 0 \\
 2 & & \\
 \hline
 17 & =8 & 1 \\
 2 & & \\
 \hline
 8 & =4 & 0 \\
 2 & & \\
 \hline
 4 & =2 & 0 \\
 2 & & \\
 \hline
 2 & =1 & 0 \\
 2 & & \\
 \hline
 1 & =0 & 1 \\
 2 & &
 \end{array}$$

The straight binary code is 10001001_2

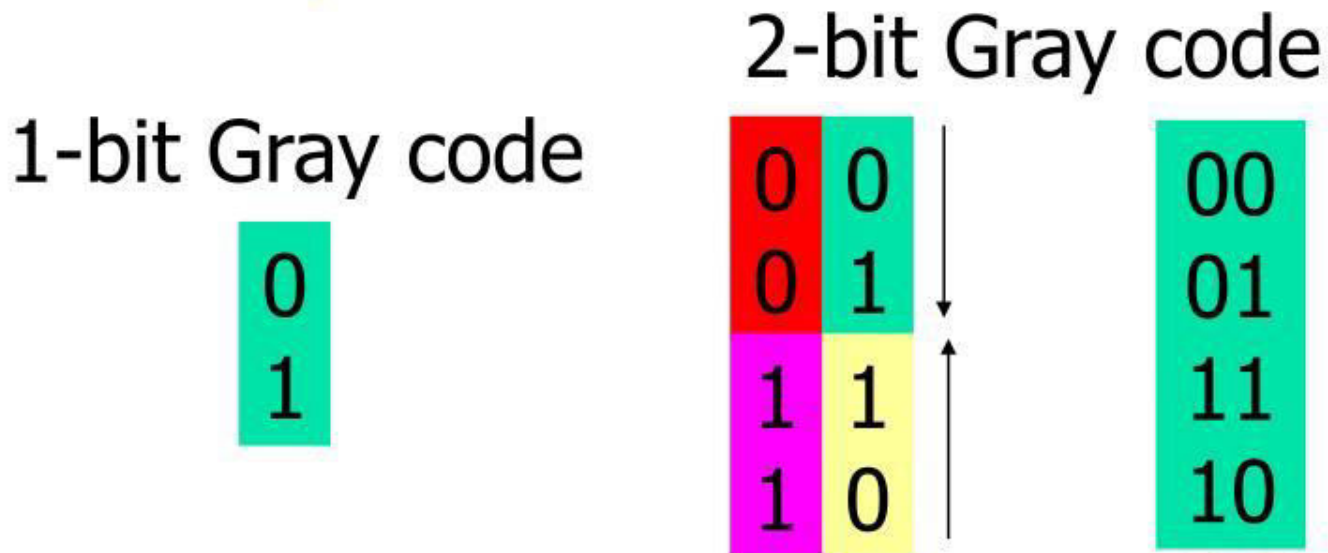
2.4 Gray Code

- In a Gray code only one bit changes between each pair of successive code words.
- Gray code is a *reflected code*. It can be defined recursively using the following rules:
 1. A 1-bit Gray code has two code words, 0 and 1.
 2. The first 2^n code words of an $(n+1)$ -bit Gray code equal the code words of an n -bit gray code, written in order with a leading 0 appended.

2.4 Gray Code

3. The last 2^n code words of an $(n+1)$ -bit Gray code equal the code words of an n -bit gray code, but written in reverse order with a leading 1 appended.

■ **Example:** For a 2-bit Gray code, $n=1$.



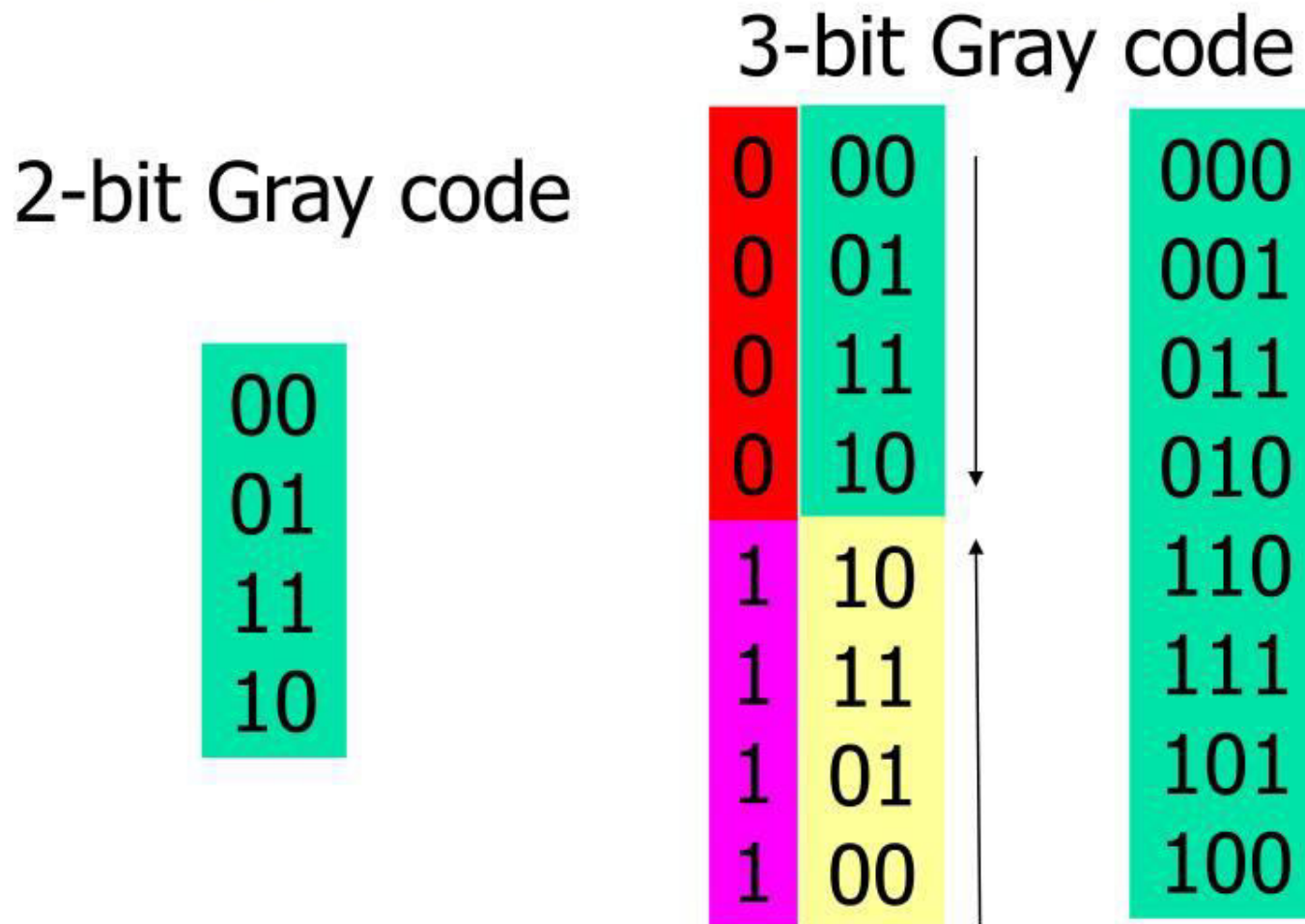
Return

Back

Next

2.4 Gray Code

- **Example:** For a 3-bit Gray code, $n=2$.



Return

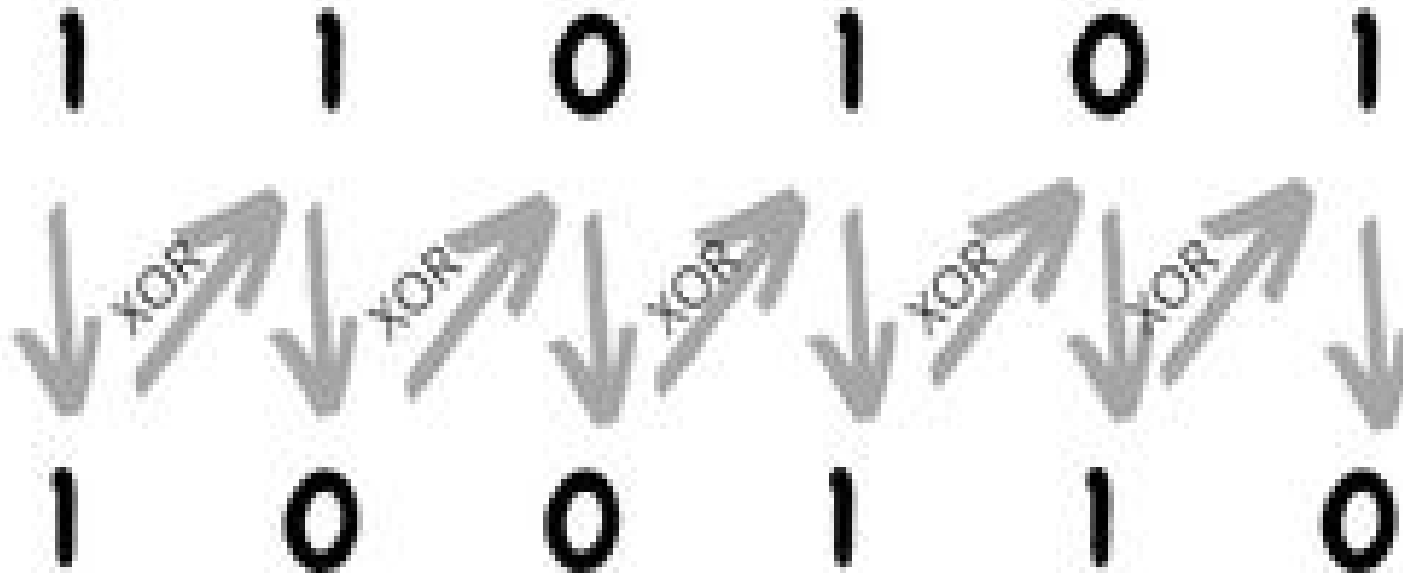
Back

Gray Code

- No weights assigned to the bit positions
- Only a single bit change from one code word to the next in sequence.
- Good— minimize the chance for error.

DECIMAL	BINARY	GRAY CODE
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Gray Code



Binary Code

Conversion from Gray to Binary

Gray Code Input				Binary Code Output			
G3	G2	G1	G0	B3	B2	B1	B0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	1	0	0	1	0
0	0	1	0	0	0	1	1
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	1
0	1	0	1	0	1	1	0
0	1	0	0	0	1	1	1
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	1
1	1	1	1	1	0	1	0
1	1	1	0	1	0	1	1
1	0	1	0	1	1	0	0
1	0	1	1	1	1	0	1
1	0	0	1	1	1	1	0
1	0	0	0	1	1	1	1

Table1

Part-1-Design Procedure

- ◆ Determine the inputs and outputs
- ◆ Assign a symbol for each
- ◆ Derive the truth table
- ◆ Get the simplified boolean expression for each output
- ◆ Draw the network diagram

Example

◆ Conversion from BCD to excess-5

INPUT				OUTPUT			
A	B	C	D	W	X	Y	Z
0	0	0	0	0	1	0	1
0	0	0	1	0	1	1	0
0	0	1	0	0	1	1	1
0	0	1	1	1	0	0	0
0	1	0	0	1	0	0	1
0	1	0	1	1	0	1	0
0	1	1	0	1	0	1	1
0	1	1	1	1	1	0	0
1	0	0	0	1	1	0	1
1	0	0	1	1	1	1	0

Example (Cont.)

$$W = A + B + CD$$

AB \ CD		C'		C	
		00	01	11	10
A'	00	0	0	1	0
	01	1	1	1	1
A	11	X	X	X	X
	10	1	1	X	X
		D'		D	D'

Groupings on the right side of the table: B' (rows 00, 01), B (rows 11, 10), and B' (rows 00, 01).

Example (Cont.)

$$X = A + B'D' + B'C' + BCD$$

AB \ CD		C'		C	
		00	01	11	10
A'	00	1	1	0	1
	01	0	0	1	0
A	11	X	X	X	X
	10	1	1	X	X
		D'		D	

Groupings for the expression $X = A + B'D' + B'C' + BCD$:

- A**: Rows 11 and 10 (A=1)
- B'D'**: Cells (00,00), (01,00), (10,00), (10,01)
- B'C'**: Cells (00,00), (01,00), (00,01), (01,01)
- BCD**: Cells (11,01), (11,11), (10,11), (10,10)

Example (Cont.)

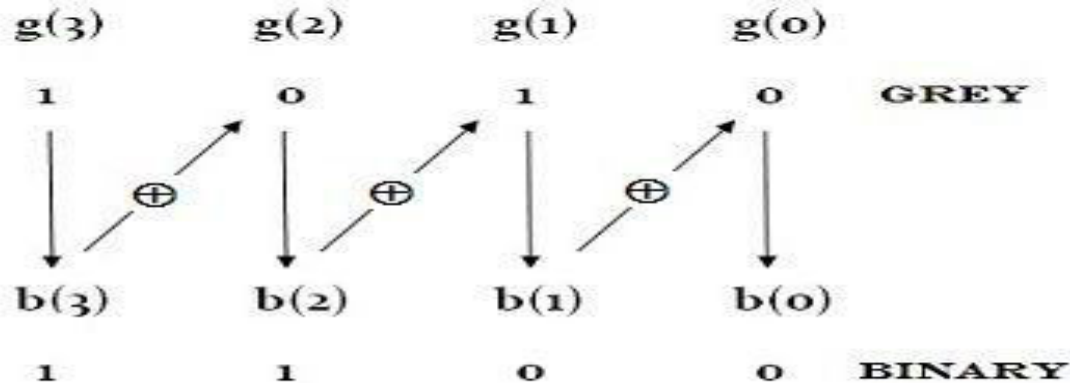
Find Y and Z

Draw the network diagram

For more details read the text Book

Grey Code to Binary Conversion

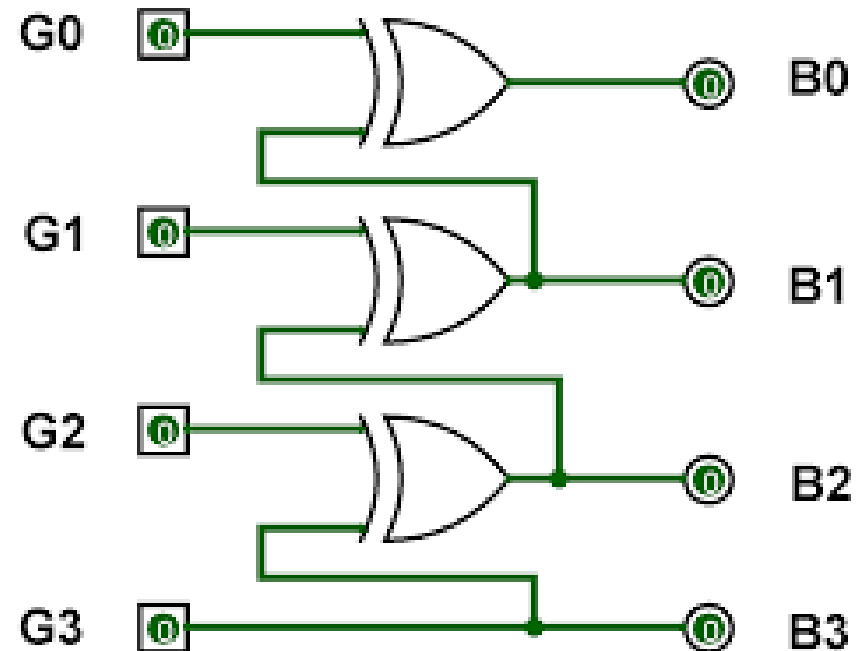
Convert the Grey code 1010 to its equivalent Binary



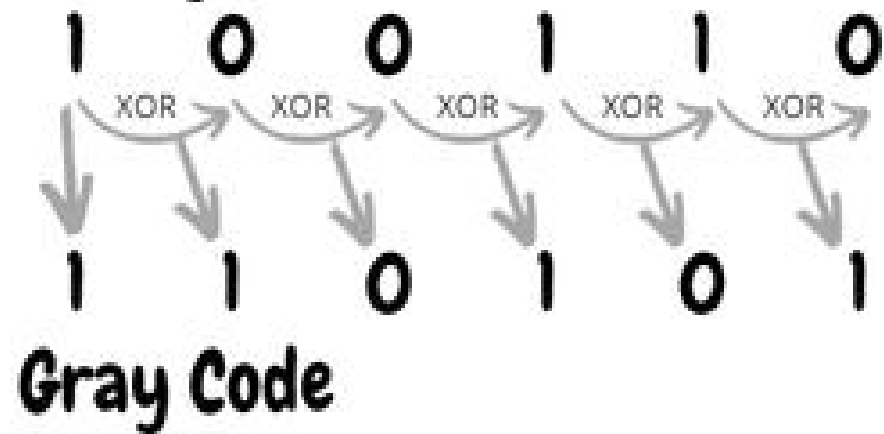
i.e

$$b(3) = g(3)$$
$$b(2) = b(3) \oplus g(2)$$
$$b(1) = b(2) \oplus g(1)$$
$$b(0) = b(1) \oplus g(0)$$

Boolean functions



Binary Code

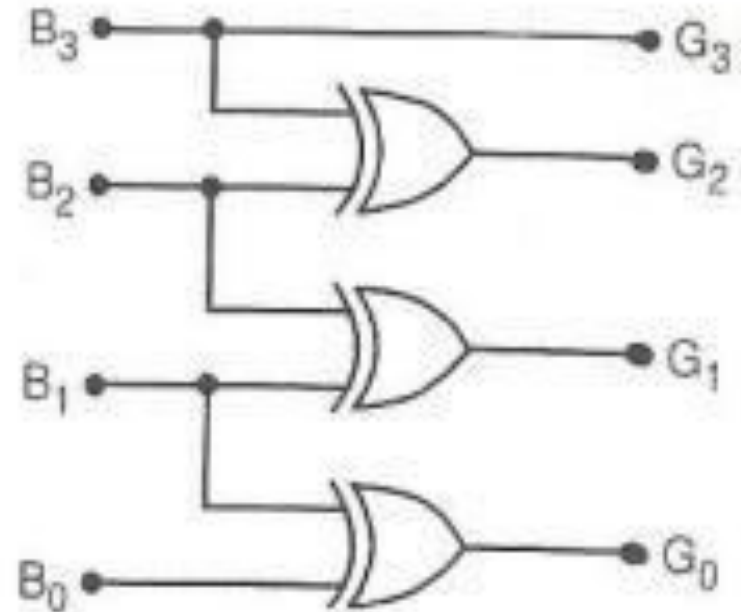


Binary to Gray Code Converter



Boolean functions

$$\begin{aligned}G_3 &= B_3 \\G_2 &= B_3 \oplus B_2 \\G_1 &= B_2 \oplus B_1 \\G_0 &= B_1 \oplus B_0\end{aligned}$$



Adders

- ◆ Essential part of every CPU
- ◆ Half adder (Ignore the carry-in bit)
 - It performs the addition of two bits
- ◆ Full adder
 - It performs the addition of three bits

Functional Blocks: Addition

- **Binary addition used frequently**
- **Addition Development:**
 - *Half-Adder (HA)*, a 2-input bit-wise addition functional block,
 - *Full-Adder (FA)*, a 3-input bit-wise addition functional block,
 - *Ripple Carry Adder*, an iterative array to perform binary addition, and
 - *Carry-Look-Ahead Adder (CLA)*, a hierarchical structure to improve performance.

Functional Block: Half-Adder

- A 2-input, 1-bit width binary adder that performs the following computations:

X	0	0	1	1
+ Y	+ 0	+ 1	+ 0	+ 1
C S	0 0	0 1	0 1	1 0

- A half adder adds two bits to produce a two-bit sum
- The sum is expressed as a sum bit, S and a carry bit, C
- The half adder can be specified as a truth table for S and C $\Rightarrow$

X	Y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Logic Simplification: Half-Adder

- The K-Map for S, C is:
- This is a pretty trivial map!
By inspection:

$$S = X \times \overline{Y} + \overline{X} \times Y = X \oplus Y$$

$$S = (X + Y) \times \overline{(X + Y)}$$

- and

$$C = X \times Y$$

$$C = \overline{\overline{(X \times Y)}}$$

- These equations lead to several implementations.

S		Y	
	0	1	1
X	1		3

C		Y	
	0	1	
X		1	3

Five Implementations: Half-Adder

- We can derive following sets of equations for a half-adder:

$$(a) \begin{aligned} S &= X \times \bar{Y} + \bar{X} \times Y \\ C &= X \times Y \end{aligned} \quad (d) \begin{aligned} \underline{S} &= (\underline{X} + \underline{Y}) \times \bar{C} \\ \bar{C} &= (\bar{X} + \bar{Y}) \end{aligned}$$

$$(b) \begin{aligned} S &= (X + Y) \times (\bar{X} + \bar{Y}) \\ C &= \underline{X \times Y} \end{aligned} \quad (e) \begin{aligned} S &= X \oplus Y \\ C &= X \times Y \end{aligned}$$

$$(c) \begin{aligned} S &= (\underline{C} + \bar{X} \times \bar{Y}) \\ C &= X \times Y \end{aligned}$$

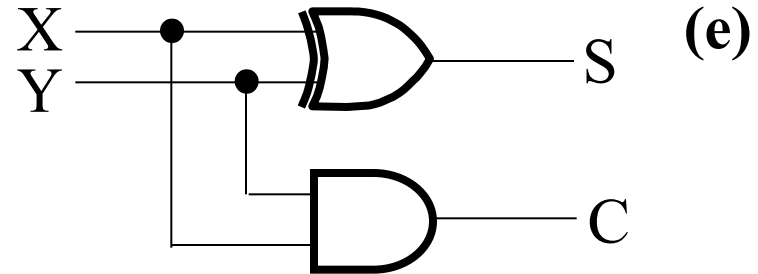
- (a), (b), and (e) are SOP, POS, and XOR implementations for S.
- In (c), the C function is used as a term in the AND-NOR implementation of S, and in (d), the $\bar{C}$ function is used in a POS term for S.

Implementations: Half-Adder

- The most common half adder implementation is:

$$S = X \oplus Y$$

$$C = X \times Y$$



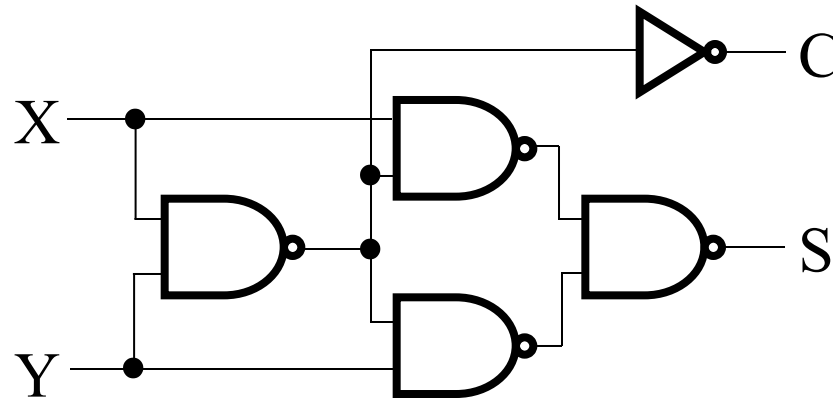
- A NAND only implementation is:

$$S = (X + Y) \times \bar{C}$$

$$= X \times \bar{C} + Y \times \bar{C}$$

$$= \overline{(X \times \bar{C}) \times (Y \times \bar{C})}$$

$$C = \overline{X \times Y}$$



Functional Block: Full-Adder

- A full adder is similar to a half adder, but includes a carry-in bit from lower stages. Like the half-adder, it computes a sum bit, S and a carry bit, C.

- For a carry-in (Z) of 0, it is the same as the half-adder:

Z	0	0	0	0
X	0	0	1	1
<u>+ Y</u>	<u>+ 0</u>	<u>+ 1</u>	<u>+ 0</u>	<u>+ 1</u>
C S	0 0	0 1	0 1	1 0

- For a carry- in (Z) of 1:

Z	1	1	1	1
X	0	0	1	1
<u>+ Y</u>	<u>+ 0</u>	<u>+ 1</u>	<u>+ 0</u>	<u>+ 1</u>
C S	0 1	1 0	1 0	1 1

Logic Optimization: Full-Adder

■ Full-Adder Truth Table:

X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

■ Full-Adder K-Map:

K-Map for Sum (S):

			Y
	0	1	3
			2
X	1		7
	4	5	6
			Z

K-Map for Carry (C):

			Y
	0	1	3
			2
X		1	7
	4	5	6
			Z

Equations: Full-Adder

- From the K-Map, we get:

$$S = X \bar{Y} \bar{Z} + \bar{X} Y \bar{Z} + \bar{X} \bar{Y} Z + X Y Z$$

$$C = X Y + X Z + Y Z$$

- The S function is the three-bit XOR function (Odd Function):

$$S = X \oplus Y \oplus Z$$

- The Carry bit C is 1 if both X and Y are 1 (the sum is 2), or if the sum is 1 and a carry-in (Z) occurs. Thus C can be re-written as:

$$C = X Y + (X \oplus Y) Z$$

- The term $X \cdot Y$ is *carry generate*.
- The term $X \oplus Y$ is *carry propagate*.

Logic diagram of full adder

$$S = X \oplus Y \oplus Z$$

$$C = XY + (X \oplus Y)Z$$

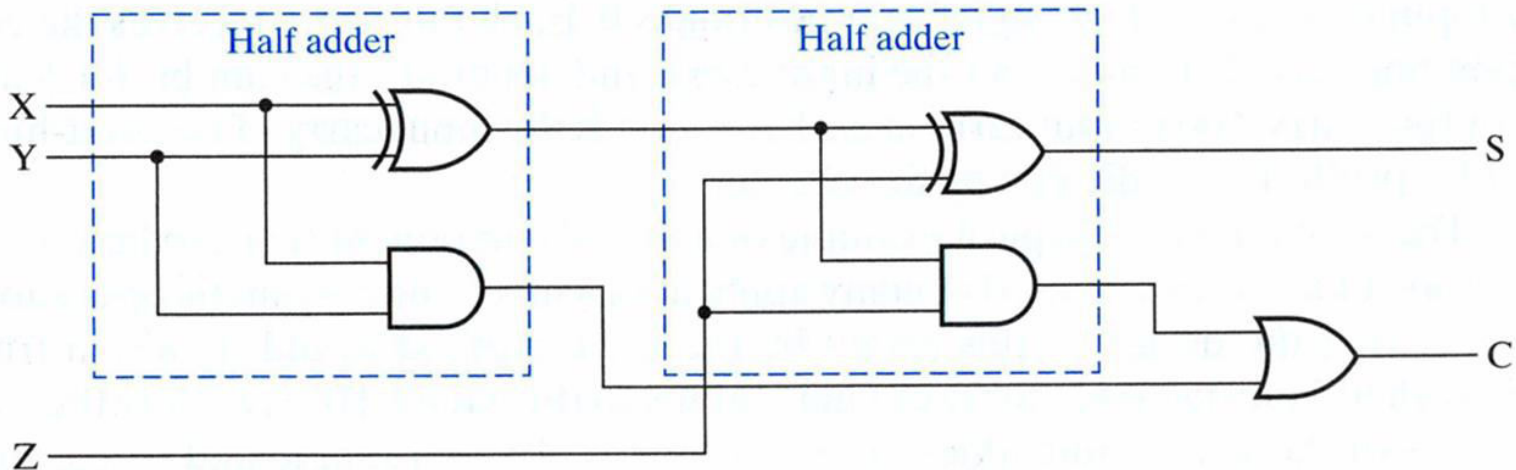
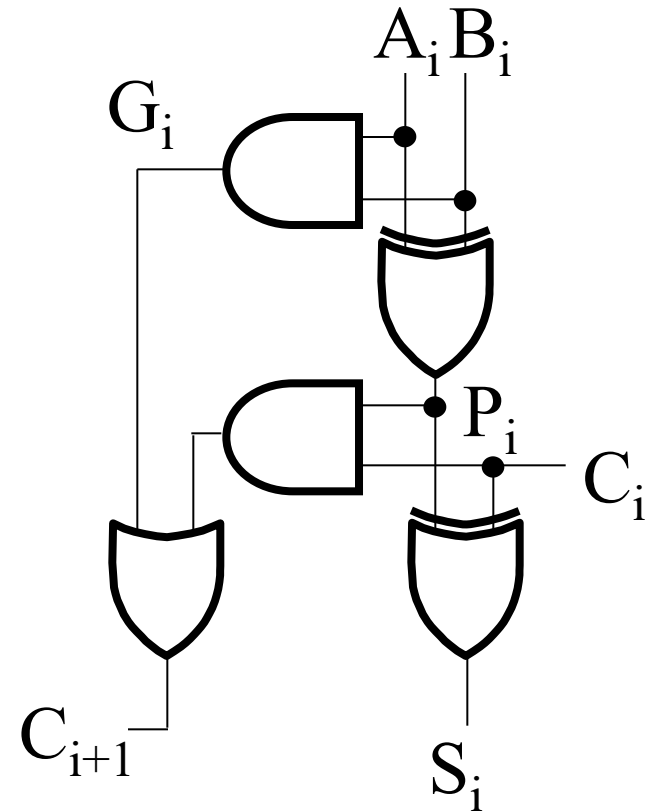


FIGURE 3-42
Logic Diagram of Full Adder

Another Implementation of full adder

- **Full Adder Schematic**
- Here X, Y, and Z, and C (from the previous pages) are A_i , B_i , C_i and C_{i+1} , respectively. Also,
 - G** = generate and
 - P** = propagate.
- **Note:** This is really a combination of a 3-bit odd function (for S) and Carry logic (for C_{i+1}):



(G = Generate) OR (P = Propagate AND C_i = Carry In)

$$C_{i+1} = G + P \cdot C_i$$

BINARY SUBTRACTION

Example: Subtract binary number 101 from 1011

(borrow)

$$\begin{array}{r} \\ \\ \\ - \\ \hline \end{array}$$

0 1 0 1 1
1 0 1 1
- 1 0 1
0 1 1 0



TEST

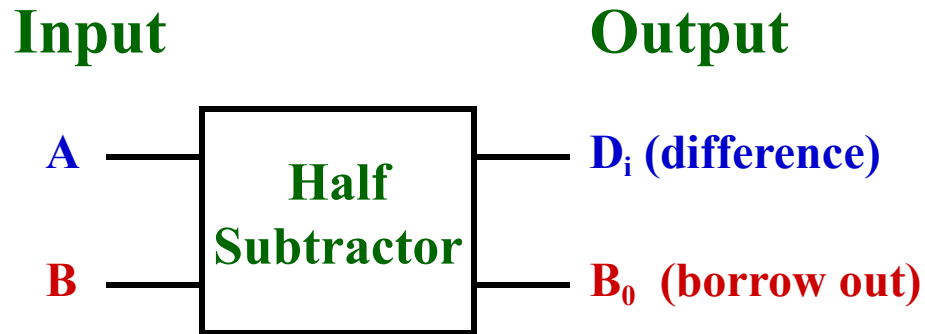
Subtract binary number 11 from 1010

$$\begin{array}{r} 01 \\ 0 \cancel{10} 0 \\ \cancel{1} \cancel{0} \cancel{1} 0 \\ - 1 \\ \hline 0 1 1 \end{array}$$

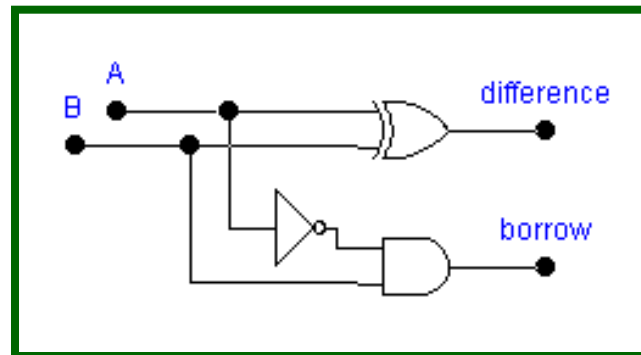
HALF SUBTRACTOR

Subtracts LSD column in binary subtraction

Logic
Symbol:



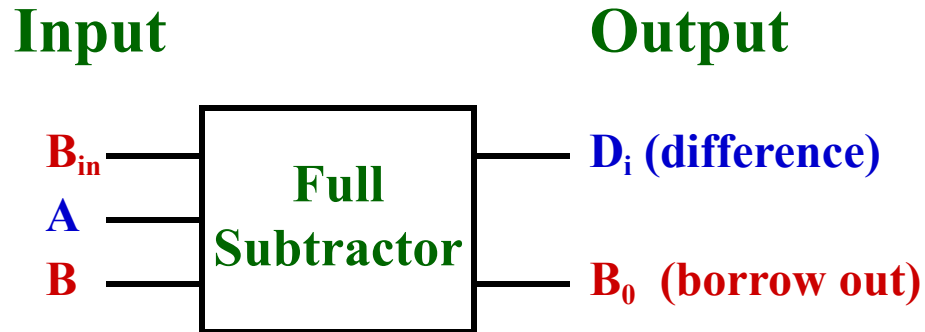
Logic
Diagram:



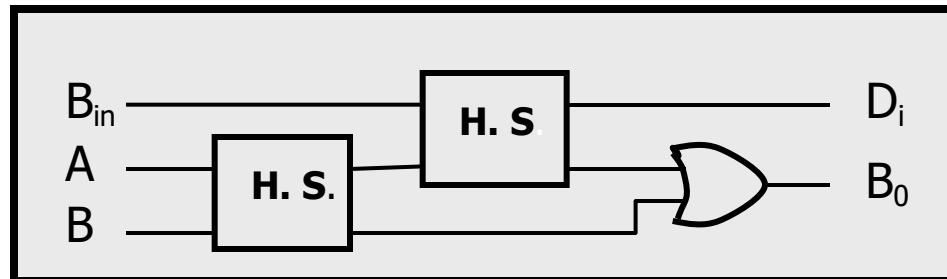
FULL SUBTRACTOR

Used for subtracting binary place values other than the 1s place

Logic
Symbol:

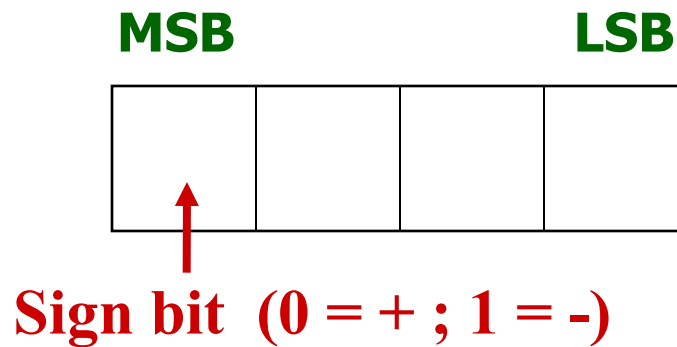


Logic
Diagram:



2s COMPLEMENT NOTATION

- 2s complement representation - widely used in microprocessors.
- Represents *sign* and *magnitude*



Decimal:	+7	+4	+1	0	-1	-4	-7
2s Complement:	0111	0100	0001	0000	1111	1100	1001

2s COMPLEMENT - CONVERSIONS

- **Converting positive numbers to 2s complement:**
 - Same as converting to binary
- **Converting negative numbers to 2s complement:**

Decimal to 2s Complement

- 4 (decimal)

0 1 0 0

1 0 1 1

- 4 = 1 1 0 0 (2s Complement)

Convert decimal
to binary

1s
complement

Add 1

2s Complement to Binary

1 1 0 0 (2s C)

0 0 1 1

0 1 0 0 (Binary)

1s
complement

Add 1

ADDING/SUBTRACTING IN 2s COMPLEMENT

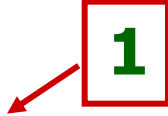
**2s complement notation makes it possible
to add and subtract signed numbers**

(Decimal)

$$\begin{array}{r} (-1) \\ + (-2) \\ \hline (-3) \end{array}$$

**2s
Complement**

$$\begin{array}{r} 1111 \\ + 1110 \\ \hline \end{array}$$

 **Discard** **11101** **2s complement**

$$\begin{array}{r} (+1) \\ + (-3) \\ \hline (-2) \end{array}$$

$$\begin{array}{r} 0001 \\ + 1101 \\ \hline 1110 \end{array} \quad \text{2s complement}$$



TEST

Add the following 2s complement numbers:

$$\begin{array}{r} (+5) \\ + (-4) \\ \hline (+1) \end{array} \quad \begin{array}{r} 0101 \\ + 1100 \\ \hline 10001 \end{array}$$

Discard

EXAMPLE 3-20 Unsigned Binary Subtraction by 2s Complement Addition

Given the two binary numbers $X = 1010100$ and $Y = 1000011$, perform the subtraction $X - Y$ and $Y - X$ using 2s complement operations. We have

$$X = 1010100$$

$$\text{2s complement of } Y = 0111101$$

$$\text{Sum} = 10010001$$

$$\text{Discard end carry } 2^7 = -\underline{10000000}$$

$$\text{Answer: } X - Y = 0010001$$

$$Y = 1000011$$

$$\text{2s complement of } X = \underline{0101100}$$

$$\text{Sum} = 1101111$$

There is no end carry.

$$\text{Answer: } Y - X = -(2\text{s complement of } 1101111) = -0010001.$$



Binary Subtractor

◆ Remember

- You need to take 2's complement to represent negative numbers
- A-B
 - ✓ Take 2's complement of B and add it to A
 - First take 1's complement and add 1

Signed Integers

- Positive numbers and zero can be represented by unsigned n -digit, radix r numbers. We need a representation for negative numbers.
- To represent a sign (+ or –) we need exactly one more bit of information (1 binary digit gives $2^1 = 2$ elements which is exactly what is needed).
- Since computers use binary numbers, by convention, the most significant bit is interpreted as a sign bit:

$$s \ a_{n-2} \ \dots \ a_2 a_1 a_0$$

where:

$s = 0$ for Positive numbers

$s = 1$ for Negative numbers

and $a_i = 0$ or 1 represent the magnitude in some form.

Signed Integer Representations

- *Signed-Magnitude* – here the $n - 1$ digits are interpreted as a positive magnitude.
- *Signed-Complement* – here the digits are interpreted as the rest of the complement of the number. There are two possibilities here:
 - *Signed 1's Complement*
 - Uses 1's Complement Arithmetic
 - *Signed 2's Complement*
 - Uses 2's Complement Arithmetic

Signed Integer Representation Example

- $r = 2, n = 3$

Number	Sign -Mag.	1's Comp.	2's Comp.
+3	011	011	011
+2	010	010	010
+1	001	001	001
+0	000	000	000
-0	100	111	—
-1	101	110	111
-2	110	101	110
-3	111	100	101
-4	—	—	100

Signed Integer Representation

□ **TABLE 3-13**
Signed Binary Numbers

Decimal	Signed 2s Complement	Signed Magnitude
+ 7	0111	0111
+ 6	0110	0110
+ 5	0101	0101
+ 4	0100	0100
+ 3	0011	0011
+ 2	0010	0010
+ 1	0001	0001
+ 0	0000	0000
− 0	—	1000
− 1	1111	1001
− 2	1110	1010
− 3	1101	1011
− 4	1100	1100
− 5	1011	1101
− 6	1010	1110
− 7	1001	1111
− 8	1000	—

Signed-Magnitude Arithmetic

- **If the parity of the two signs is 0:**
 1. Add the magnitudes.
 2. Check for overflow (a carry out of the MSB)
 3. The sign of the result is the same as the sign of the first operand.
- **If the parity of the two signs is 1:**
 1. Subtract the second magnitude from the first.
 2. If a borrow occurs:
 - take the two's complement of result
 - and make the result sign the complement of the sign of the first operand.
 3. Overflow will never occur.

Sign-Magnitude Arithmetic Examples

■ **Example 1:** **0010**
 +0101

■ **Example 2:** **0010**
 +1101

■ **Example 3:** **1010**
 - 0101

Signed-Complement Arithmetic

- Addition:

1. Add the numbers including the sign bits, discarding a carry out of the sign bits (2's Complement), or using an end-around carry (1's Complement).
2. If the sign bits were the same for both numbers and the sign of the result is different, an overflow has occurred.
3. The sign of the result is computed in step 1.

- Subtraction:

Form the complement of the number you are subtracting and follow the rules for addition.

Signed 2's Complement Examples

- **Example 1:**
$$\begin{array}{r} 1101 \\ + \underline{0011} \end{array}$$

- **Example 2:**
$$\begin{array}{r} 1101 \\ - \underline{0011} \end{array}$$

Signed Binary Addition

EXAMPLE 3-21 Signed Binary Addition Using 2s Complement

$$\begin{array}{rclclclcl} +6 & 00000110 & -6 & 11111010 & +6 & 00000110 & -6 & 11111010 \\ +\underline{13} & \underline{00001101} & +\underline{13} & \underline{00001101} & -\underline{13} & \underline{11110011} & -\underline{13} & \underline{11110011} \\ +19 & 00010011 & +7 & 00000111 & -7 & 11111001 & -19 & 11101101 \end{array}$$

In each of the four cases, the operation performed is addition, including the sign bits. Any carry out of the sign bit position is discarded, and negative results are automatically in 2s complement form. ■

Signed Binary Subtraction

EXAMPLE 3-22 Signed Binary Subtraction Using 2s Complement

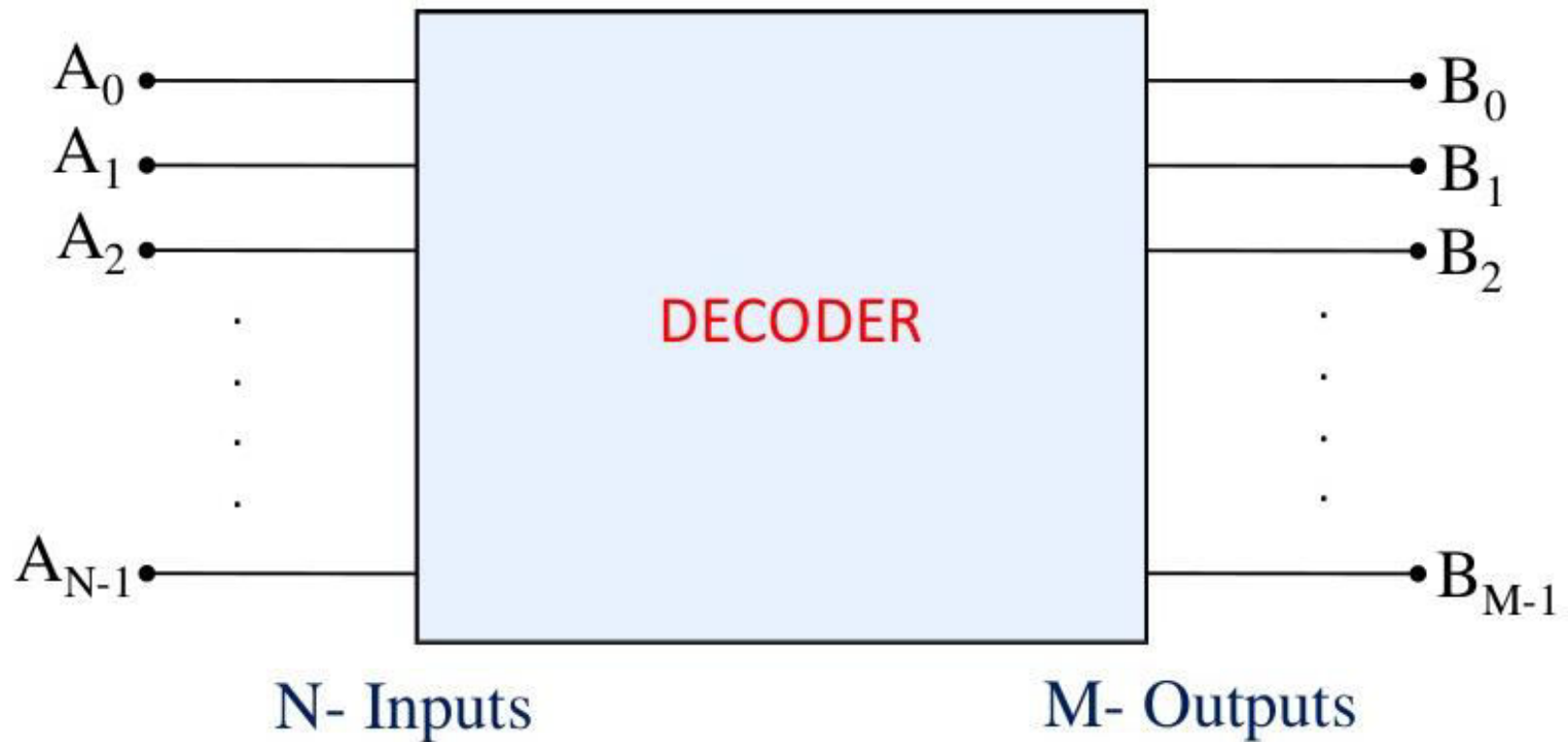
-6	11111010	11111010	$+6$	00000110	00000110
$-(-13)$	-11110011	$+00001101$	$-(-13)$	-11110011	$+00001101$
$\hline +7$	$\hline$	$\hline 00000111$	$\hline +19$	$\hline$	$\hline 00010011$

The end carry is discarded.

DECODER

- A decoder is a combinational circuit.
- A decoder accepts a set of inputs that represents a binary number and activates only that output corresponding to the input number. All other outputs remain inactive.
- Fig. 1 shows the block diagram of decoder with 'N' inputs and 'M' outputs.
- There are 2^N possible input combinations, for each of these input combination only one output will be HIGH (active) all other outputs are LOW
- Some decoder have one or more ENABLE (E) inputs that are used to control the operation of decoder.

BLOCK DIAGRAM OF DECODER



Only one output is High for each input

Fig. 1

2 to 4 Line Decoder:

- Block diagram of 2 to 4 decoder is shown in fig. 2
- A and B are the inputs. (No. of inputs =2)
- No. of possible input combinations: $2^2=4$
- No. of Outputs : $2^2=4$, they are indicated by D_0 , D_1 , D_2 and D_3
- From the Truth Table it is clear that each output is “1” for only specific combination of inputs.

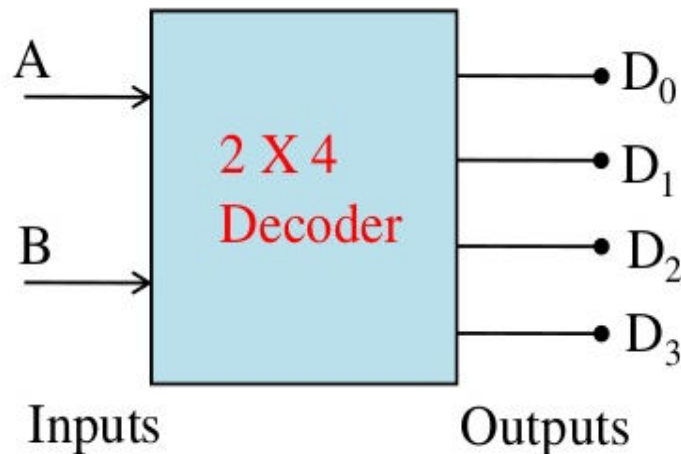


Fig. 2

TRUTH TABLE

INPUTS		OUTPUTS			
A	B	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

BOOLEAN EXPRESSION:

From Truth Table

$$D_0 = \bar{A} \bar{B}$$

$$D_1 = \bar{A} B$$

$$D_2 = A \bar{B}$$

$$D_3 = AB$$

LOGIC DIAGRAM:

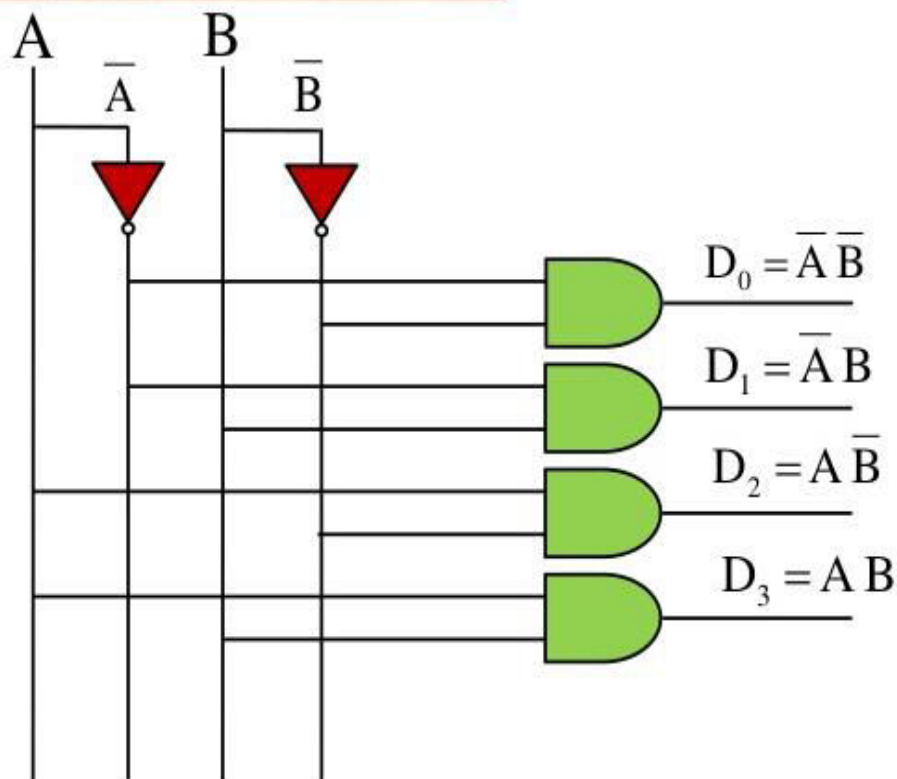
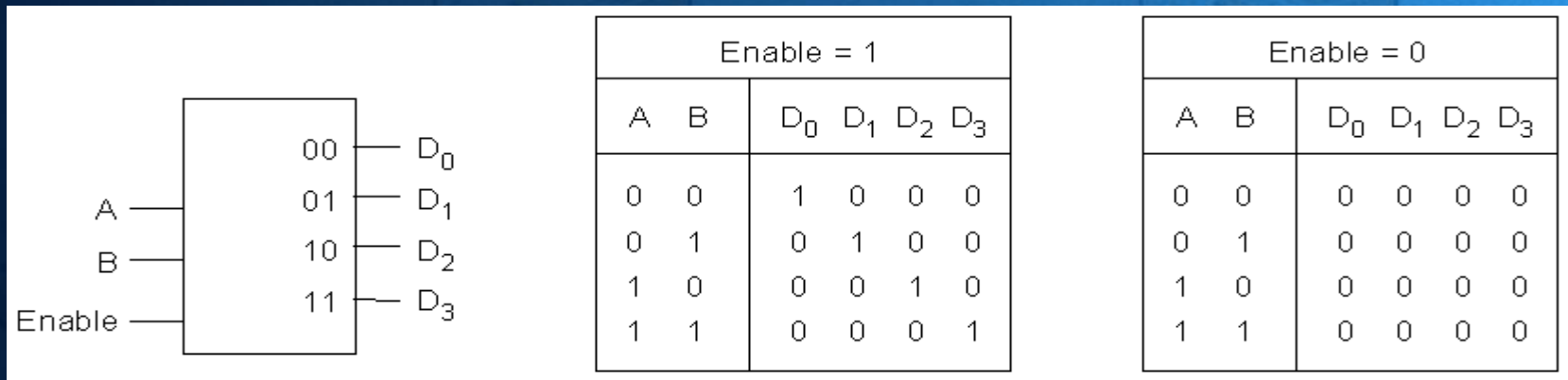


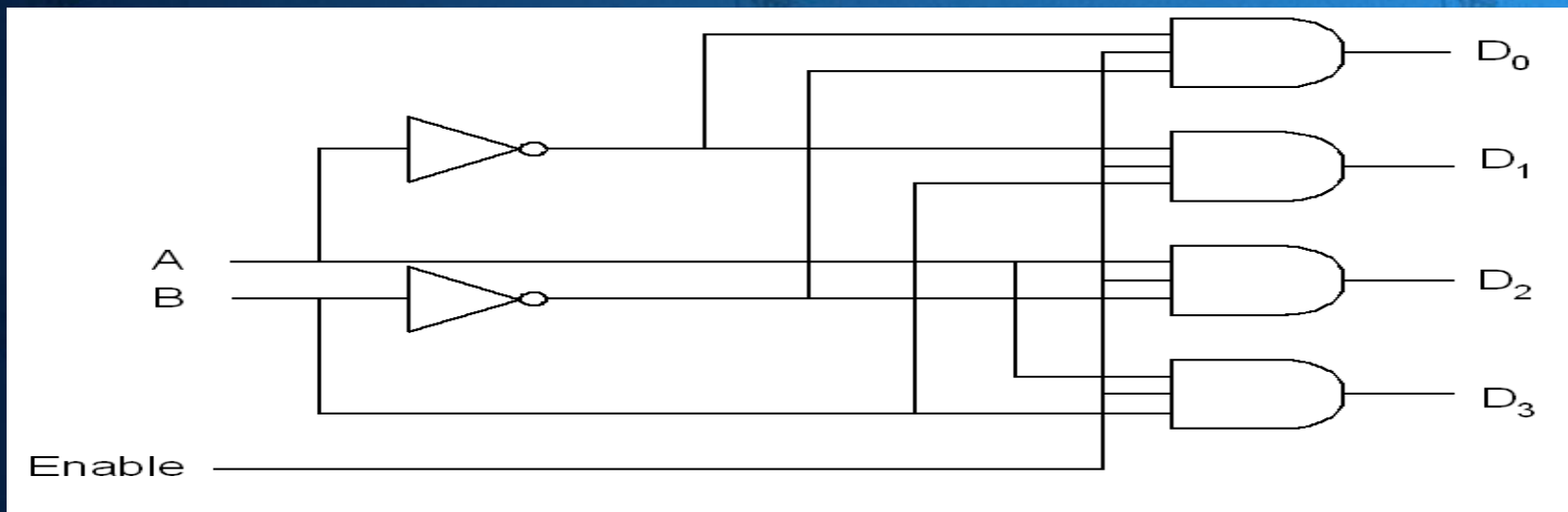
Fig. 3

Decoder

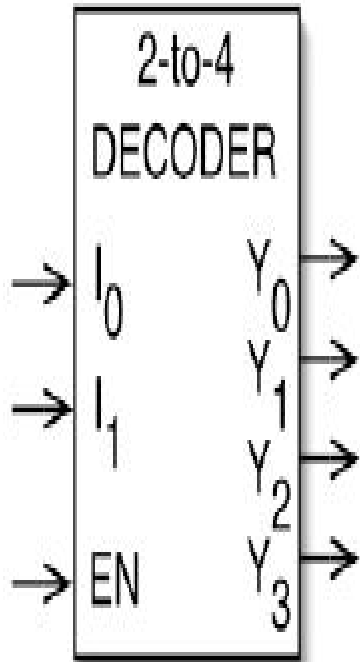
- ◆ **n by 2^n decoder**
 - **Converts information from n input lines into 2^n output lines**
 - ◆ **Example - 2x4 Decoder, 3x8 Decoder, 4x16 Decoder**
- ## 2x4 Decoder



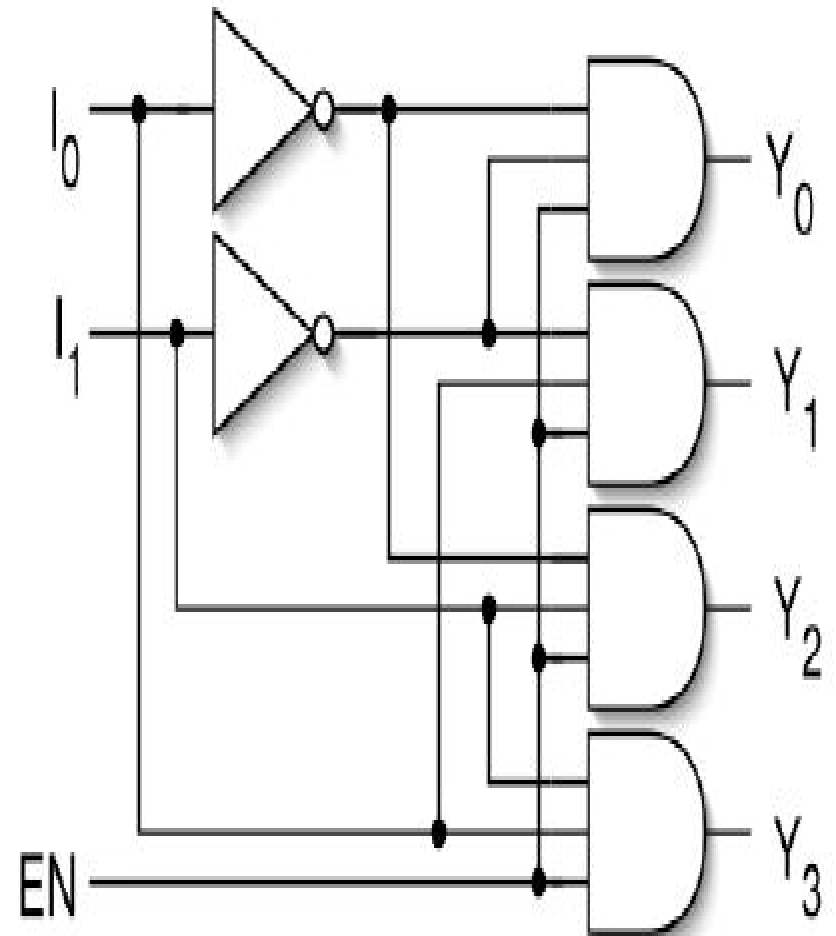
Internal Structure of 2x4 Decoder



Another View



EN	I_1	I_0	Y_3	Y_2	Y_1	Y_0
0	x	x	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0



3 to 8 Line Decoder:

- Block diagram of 3 to 8 decoder is shown in fig. 4
- A , B and C are the inputs. (No. of inputs =3)
- No. of possible input combinations: $2^3=8$
- No. of Outputs : $2^3=8$, they are indicated by D_0 to D_7
- From the Truth Table it is clear that each output is “1” for only specific combination of inputs.

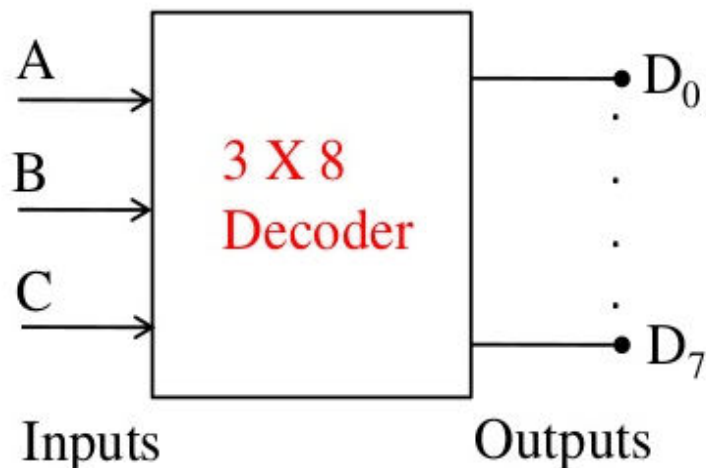


Fig. 4

TRUTH TABLE FOR 3 X 8 DECODER:

INPUTS			OUTPUTS								
A	B	C	D0	D1	D2	D3	D4	D5	D6	D7	
0	0	0	1	0	0	0	0	0	0	0	$D_0 = \bar{A} \bar{B} \bar{C}$
0	0	1	0	1	0	0	0	0	0	0	$D_1 = \bar{A} \bar{B} C$
0	1	0	0	0	1	0	0	0	0	0	$D_2 = \bar{A} B \bar{C}$
0	1	1	0	0	0	1	0	0	0	0	$D_3 = \bar{A} B C$
1	0	0	0	0	0	0	1	0	0	0	$D_4 = A \bar{B} \bar{C}$
1	0	1	0	0	0	0	0	1	0	0	$D_5 = A \bar{B} C$
1	1	0	0	0	0	0	0	0	1	0	$D_6 = A B \bar{C}$
1	1	1	0	0	0	0	0	0	0	1	$D_7 = A B C$

LOGIC DIAGRAM OF 3 X 8 DECODER:

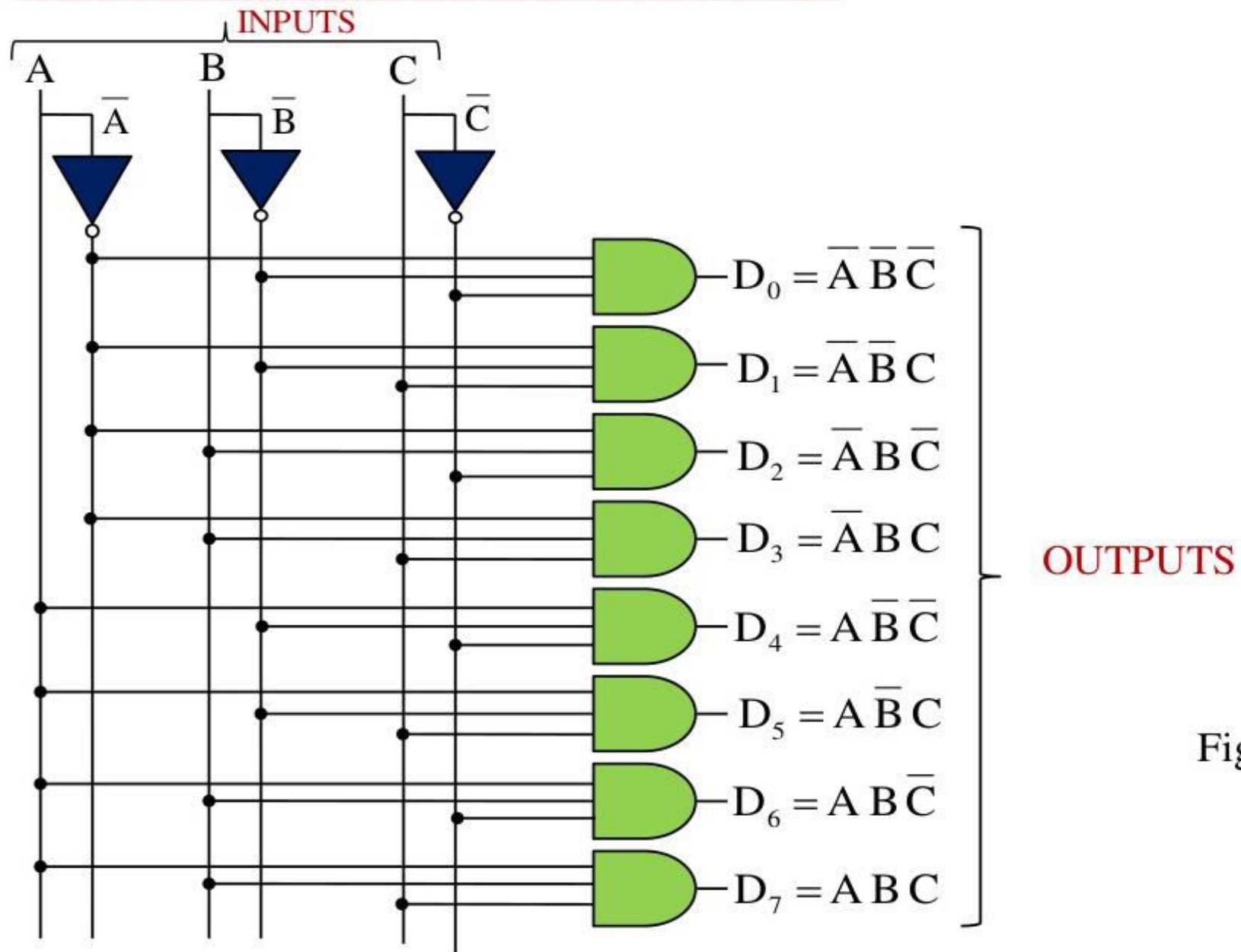


Fig. 5

EXPANSION OF DECODERS:

The number of lower order Decoder for implementing higher order Decoder can be find as

No. of lower order required = m_2/m_1

Where, m_1 =No. of Outputs of lower order Decoder

m_2 =No. of Outputs of higher order Decoder

3 x 8 Decoder From 2 x 4 Decoder:

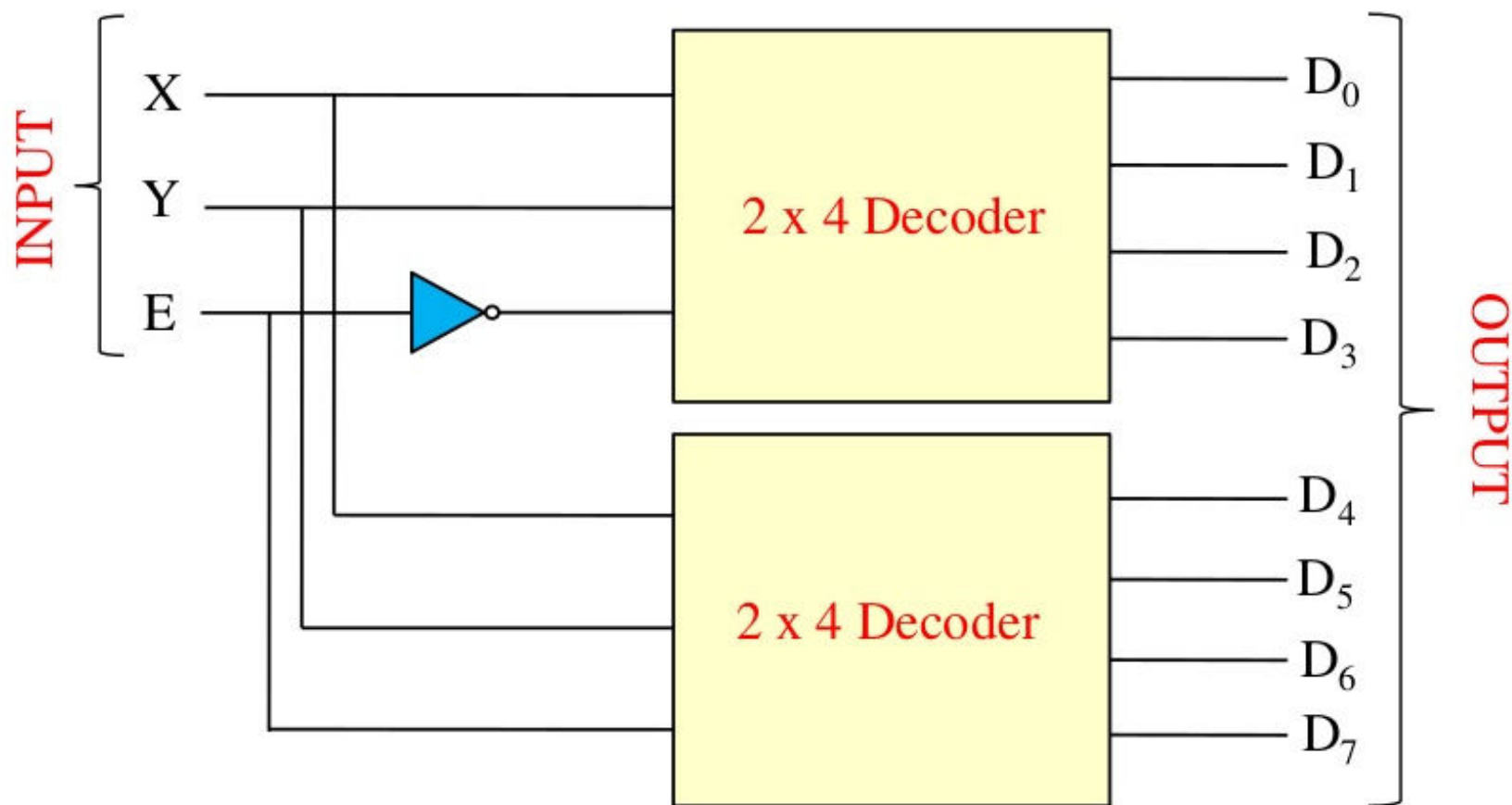


Fig. 6

4x16 Decoder

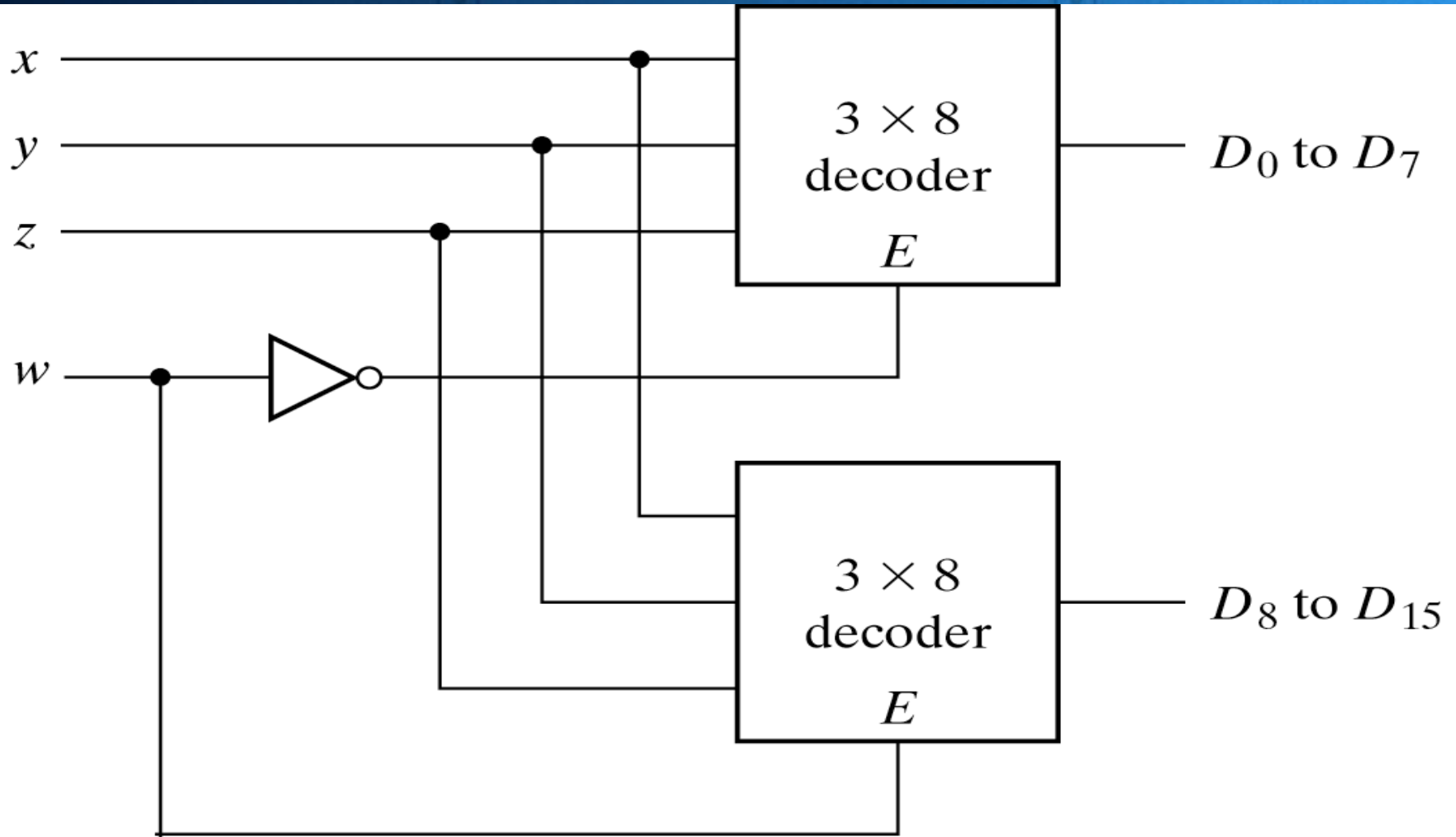


Fig. 4-20 4×16 Decoder Constructed with Two 3×8 Decoders

Decoder applications

- Microprocessor memory systems
 - | selecting different banks of memory
- Microprocessor input/output systems
 - | selecting different devices
- Microprocessor instruction decoding
 - | enabling different functional units
- Memory chips
 - | enabling different rows of memory depending on address
- Lots of other applications

Example: Implement the following multiple output function using a suitable Decoder.

$$f_1(A, B, C) = \sum m(0, 4, 7) + d(2, 3)$$

$$f_2(A, B, C) = \sum m(1, 5, 6)$$

$$f_3(A, B, C) = \sum m(0, 2, 4, 6)$$

Solution: f_1 consists of don't care conditions. So we consider them to be logic 1.

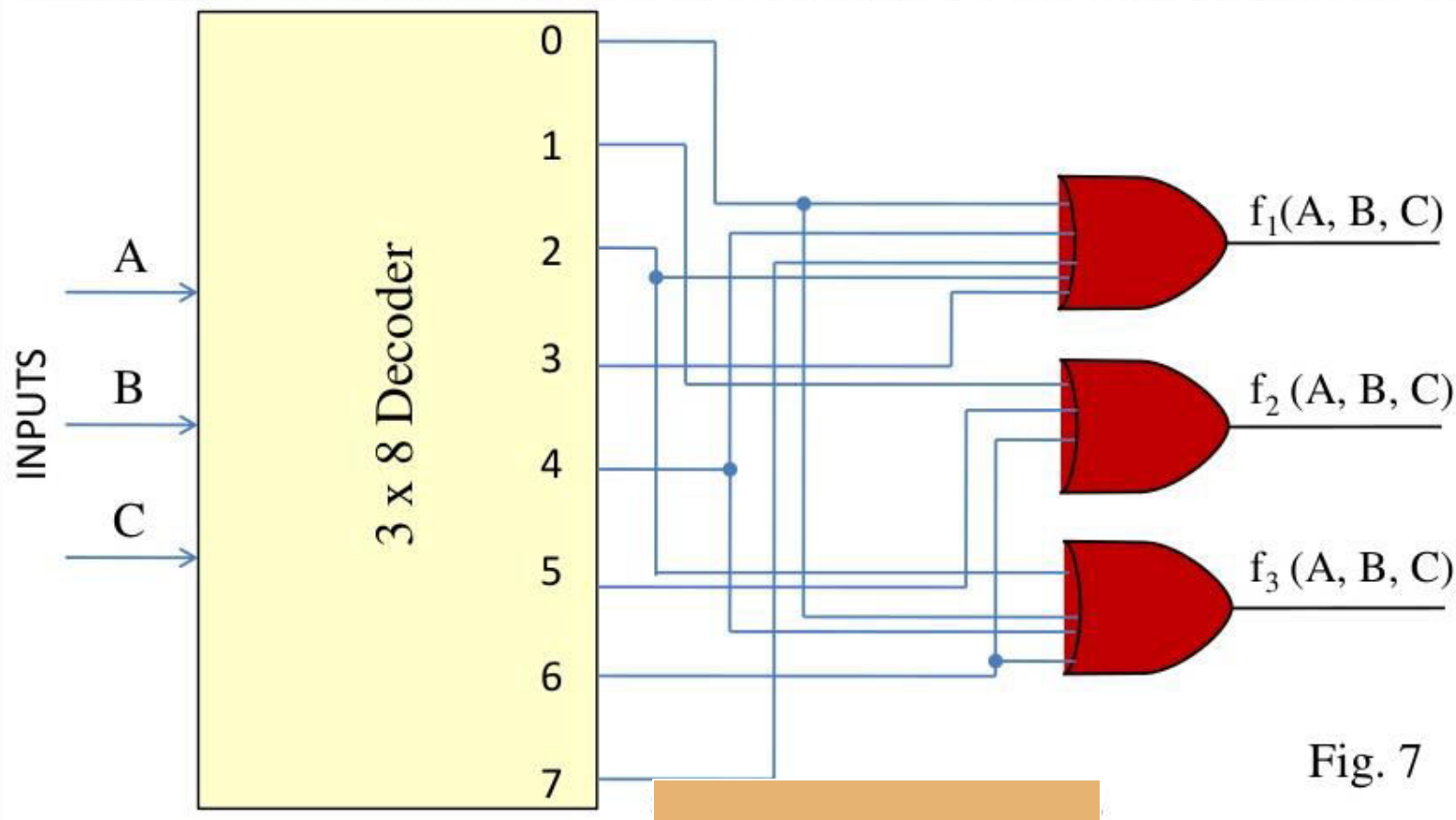


Fig. 7

EXAMPLE: Implement the following Boolean function using suitable Decoder.

$$f_1(x,y,z) = \sum m(1,5,7)$$

$$f_2(x,y,z) = \sum m(0,3)$$

$$f_3(x,y,z) = \sum m(2,4,5)$$

Solution:

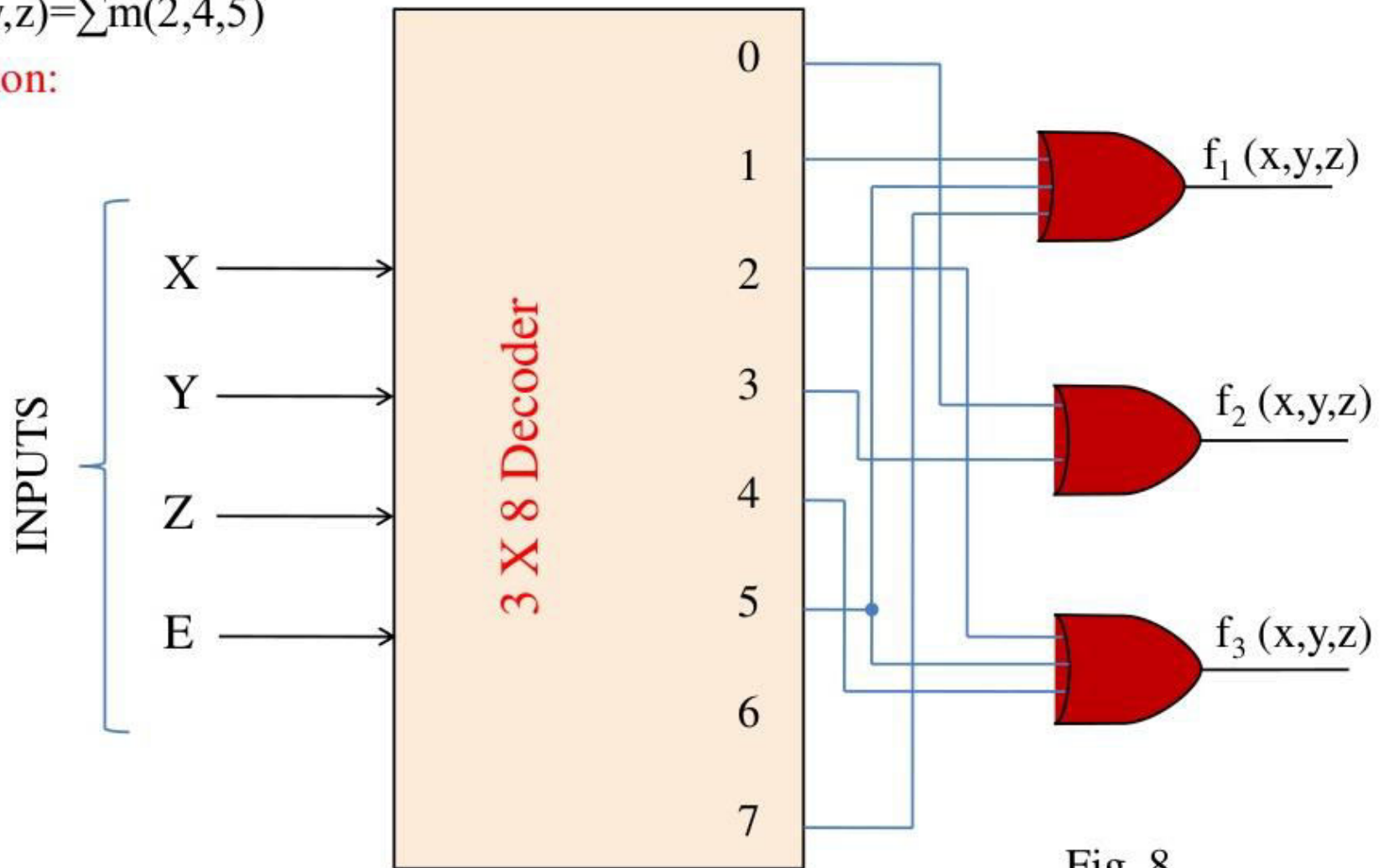


Fig. 8

EXAMPLE: A combinational circuit is defined by the following Boolean function. Design circuit with a Decoder and external gate.

$$F_1(x, y, z) = \bar{x} \bar{y} \bar{z} + x z$$

$$F_2(x, y, z) = x y \bar{z} + \bar{x} z$$

SOLUTION: STEP 1: Write the given function F_1 in SOP form

$$F_1(x, y, z) = \bar{x} \bar{y} \bar{z} + (y + \bar{y}) x z$$

$$F_1(x, y, z) = \bar{x} \bar{y} \bar{z} + x y z + x \bar{y} z$$

$$F_1(x, y, z) = \Sigma m(0, 5, 7)$$

$$F_2(x, y, z) = x y \bar{z} + \bar{x} z$$

$$F_2(x, y, z) = x y \bar{z} + (y + \bar{y}) \bar{x} z$$

$$F_2(x, y, z) = x y \bar{z} + \bar{x} y z + \bar{x} \bar{y} z$$

$$F_2(x, y, z) = \Sigma m(1, 3, 6)$$

Boolean Function using Decoder:

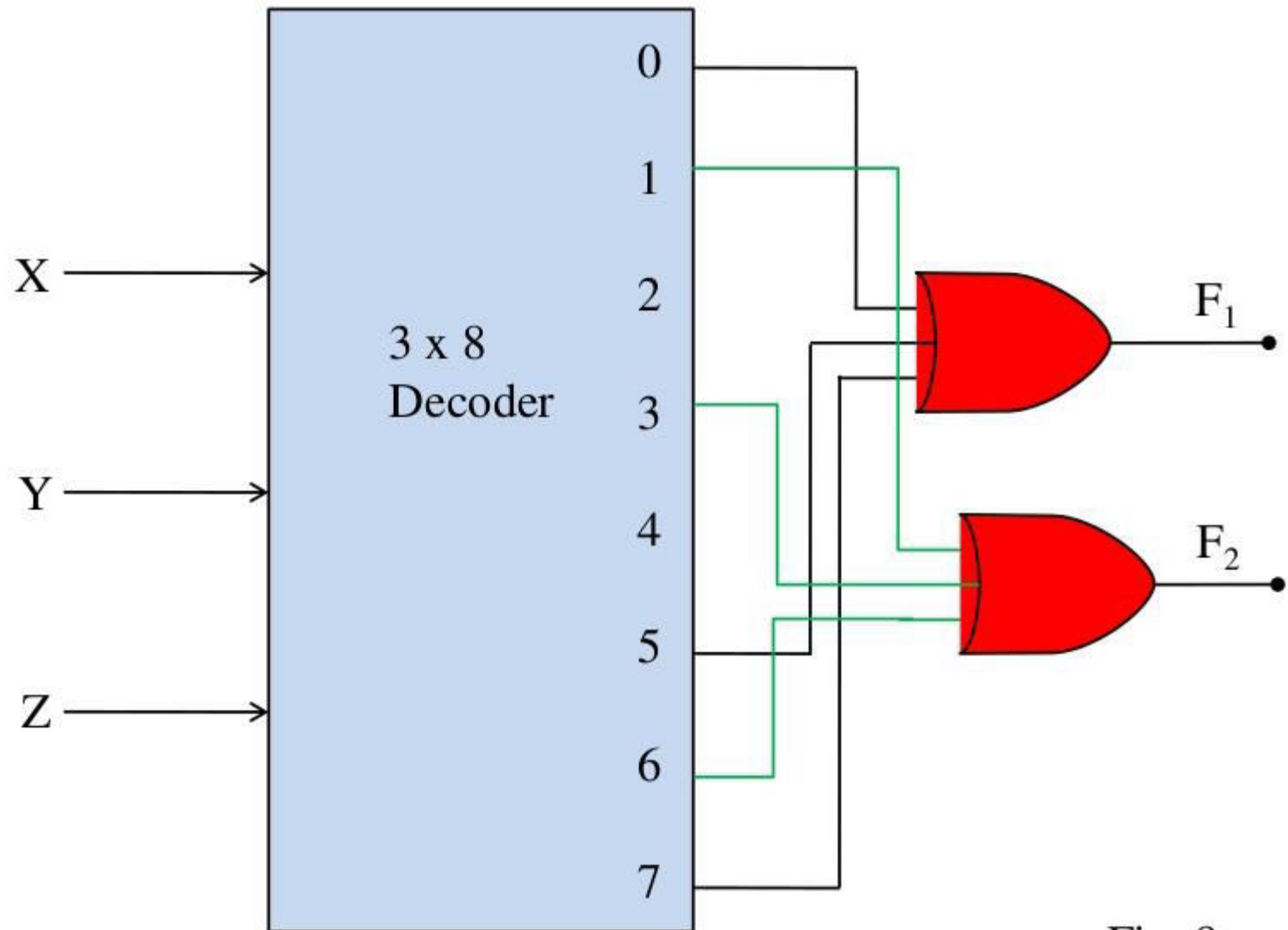


Fig. 9

3x8 Decoder

From course text book

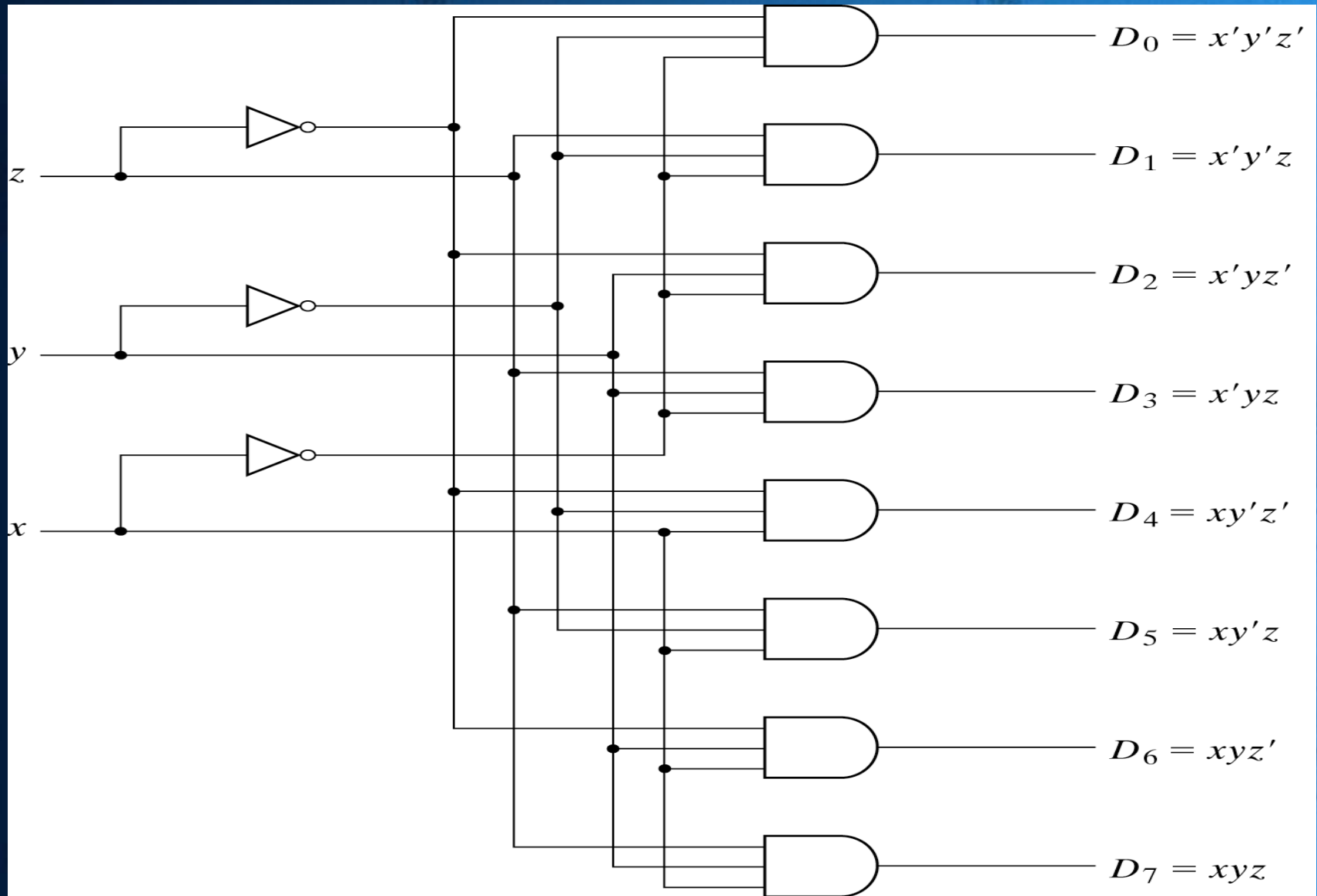
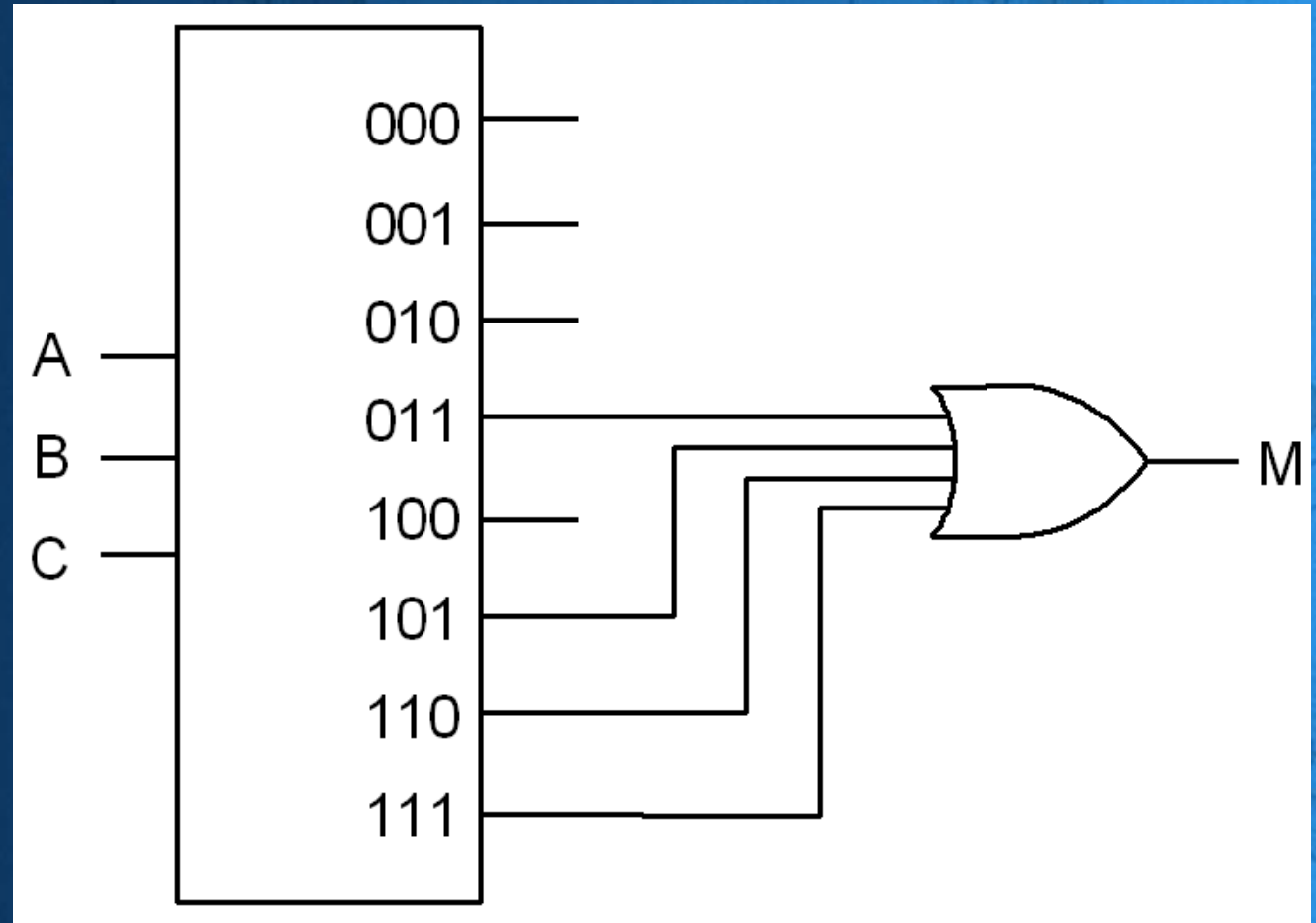


Fig. 4-18 3-to-8-Line Decoder

Example

A	B	C	M
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



Full Adder with Decoder

A_i	B_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

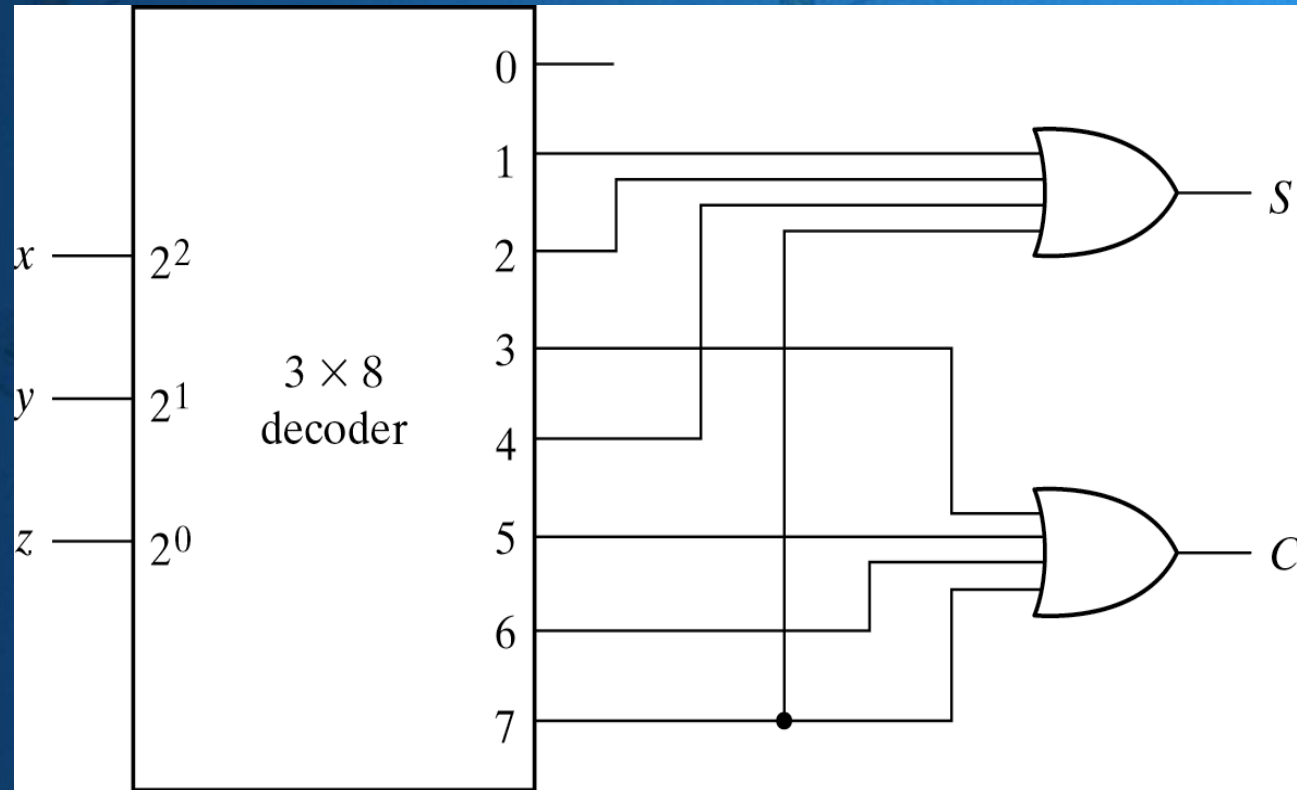


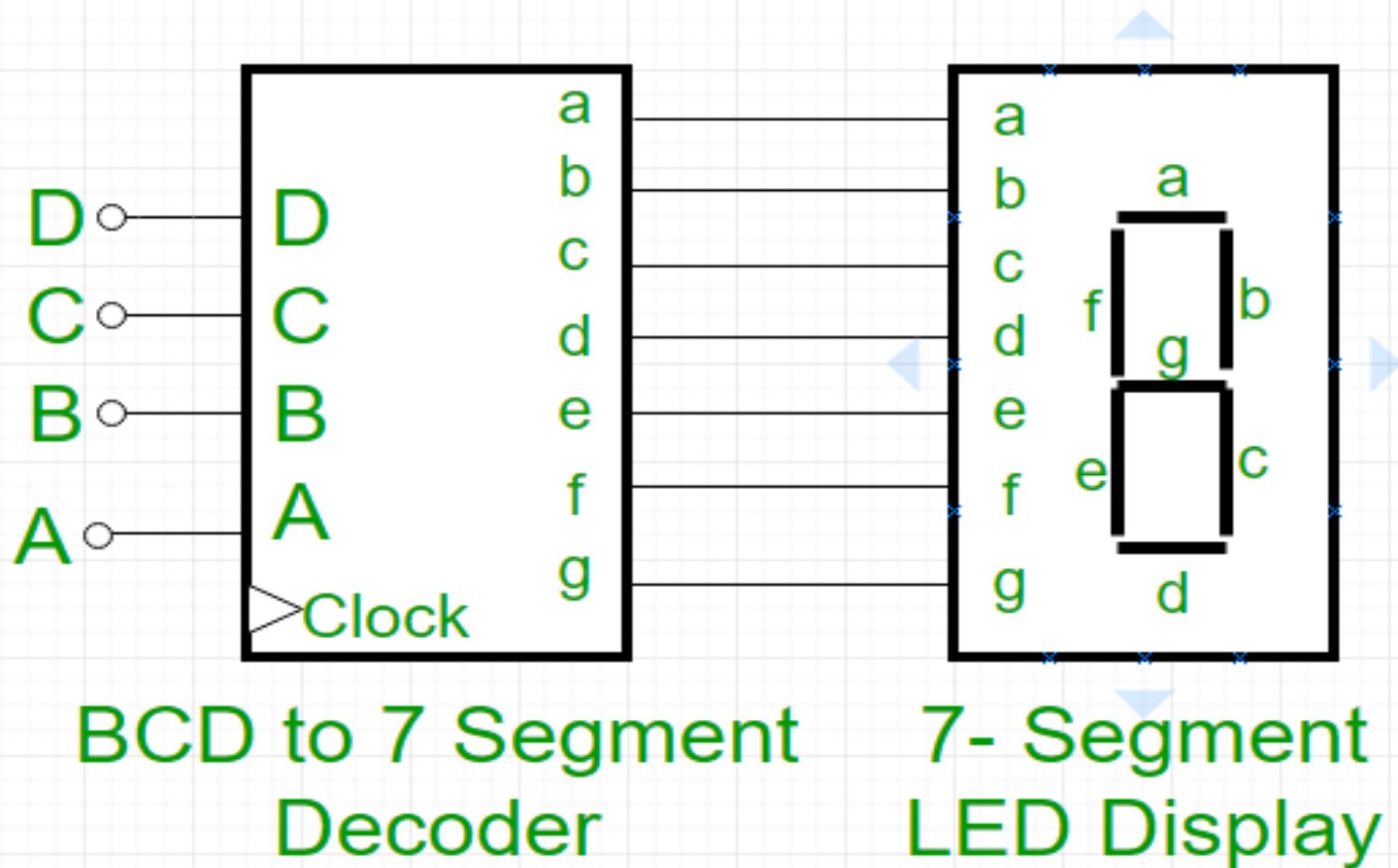
Fig. 4-21 Implementation of a Full Adder with a Decoder

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_i B_i' C_i + A_i' B_i C_i + A_i B_i C_i$$

Example 3 – BCD to 7-Segment Display

Display digits 0–9 on a 7-segment LED using a BCD-to-7-segment decoder (7447).



Solution:

Inputs: BCD (4 bits).

Decoder outputs: 7 signals (a,b,c,d,e,f,g).

For input 0101 (BCD = 5), decoder lights up segments a,c,d,f,g

Hence, the digit “5” is displayed.

ENCODER

- An Encoder is a combinational logic circuit.
- It performs the inverse operation of Decoder.
- The opposite process of decoding is known as Encoding.
- An Encoder converts an active input signal into a coded output signal.
- Block diagram of Encoder is shown in Fig.10. It has 'M' inputs and 'N' outputs.
- An Encoder has 'M' input lines, only one of which is activated at a given time, and produces an N-bit output code, depending on which input is activated.

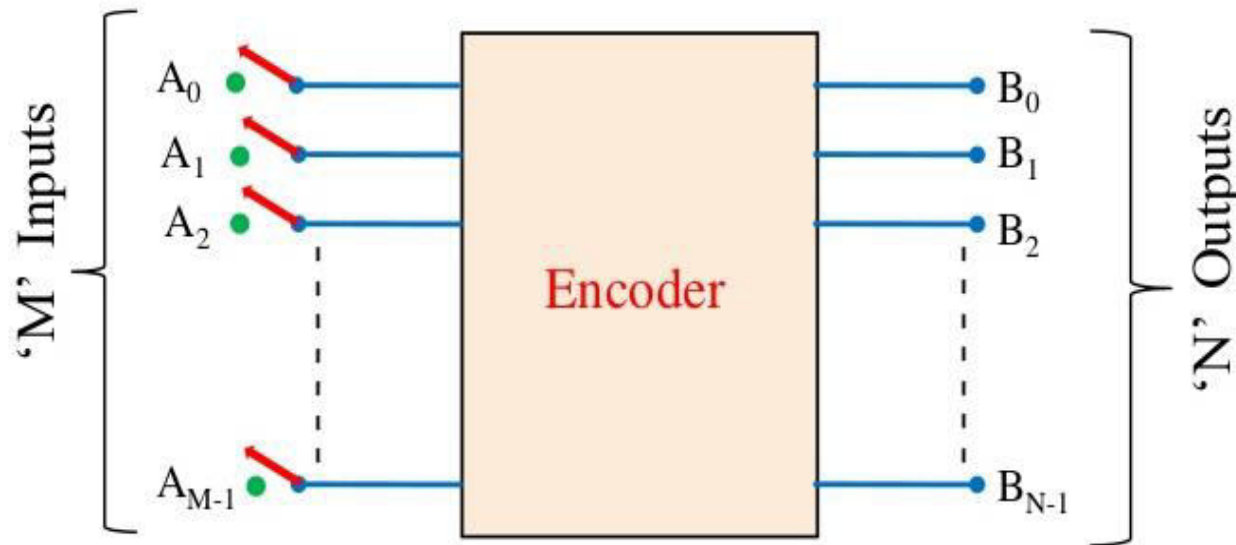


Fig. 10

- Encoders are used to translate the rotary or linear motion into a digital signal.
- The difference between Decoder and Encoder is that Decoder has Binary Code as an input while Encoder has Binary Code as an output.
- Encoder is an Electronics device that converts the analog signal to digital signal such as BCD Code.
- **Types of Encoders**
 - i. Priority Encoder
 - ii. Decimal to BCD Encoder
 - iii. Octal to Binary Encoder
 - iv. Hexadecimal to Binary Encoder

ENCODER

$$M=4$$

$$M=2^2$$

$$M=2^N$$

'M' is the input and

'N' is the output

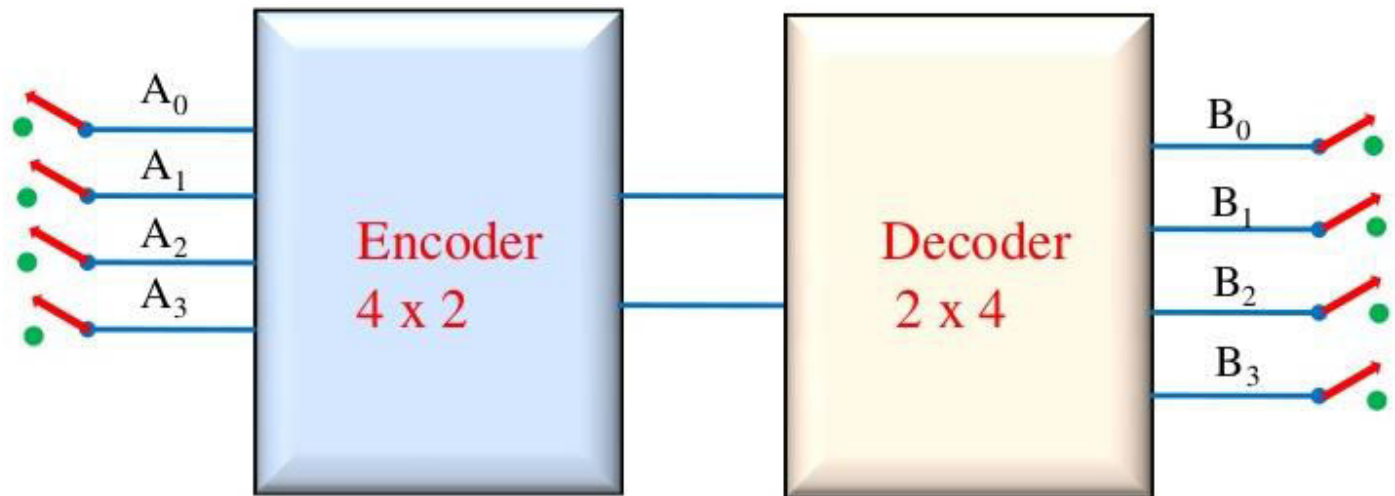


Fig. 11

ENCODER

$$M=4$$

$$M=2^2$$

$$M=2^N$$

'M' is the input and
'N' is the output

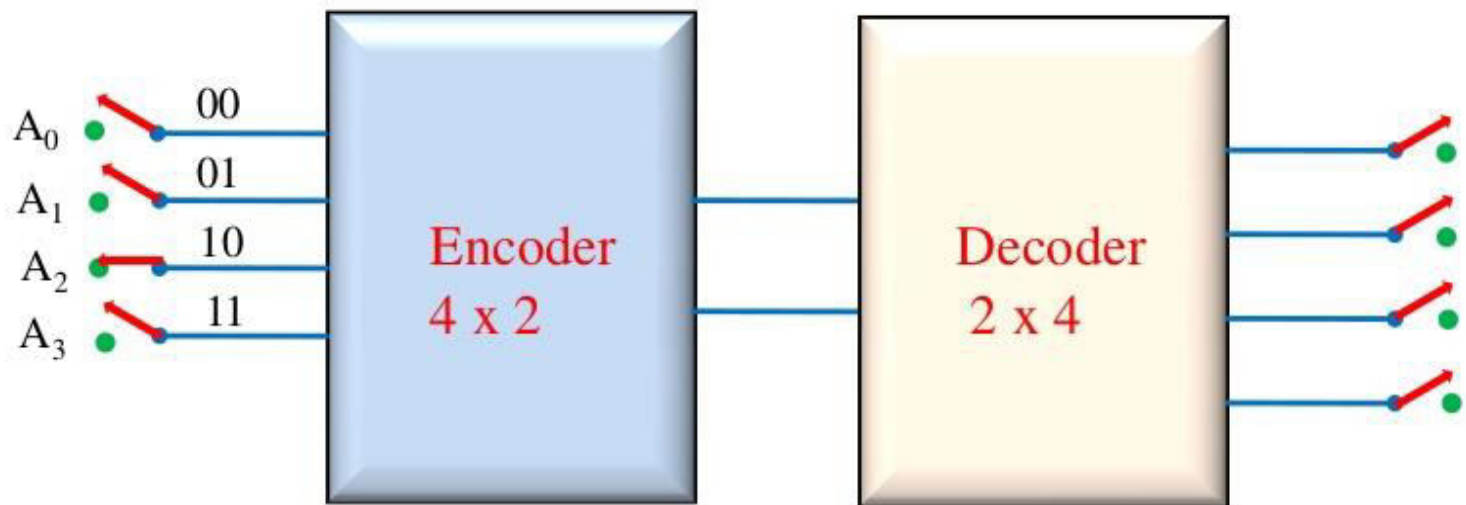


Fig. 12

ENCODER

$$M=4$$

$$M=2^2$$

$$M=2^N$$

'M' is the input and
'N' is the output

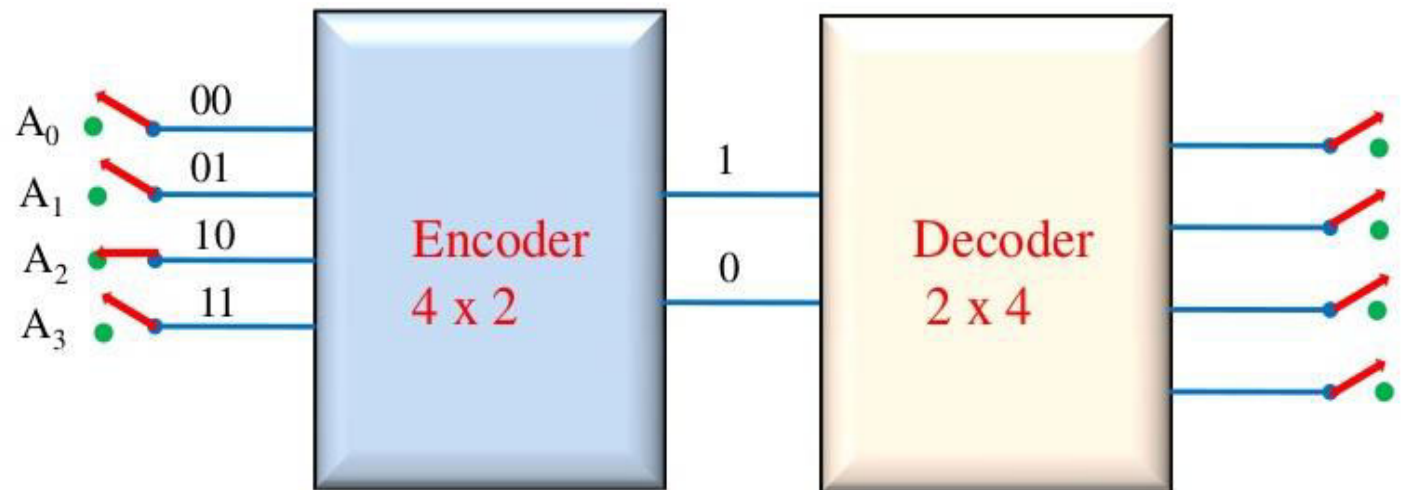


Fig. 13

ENCODER

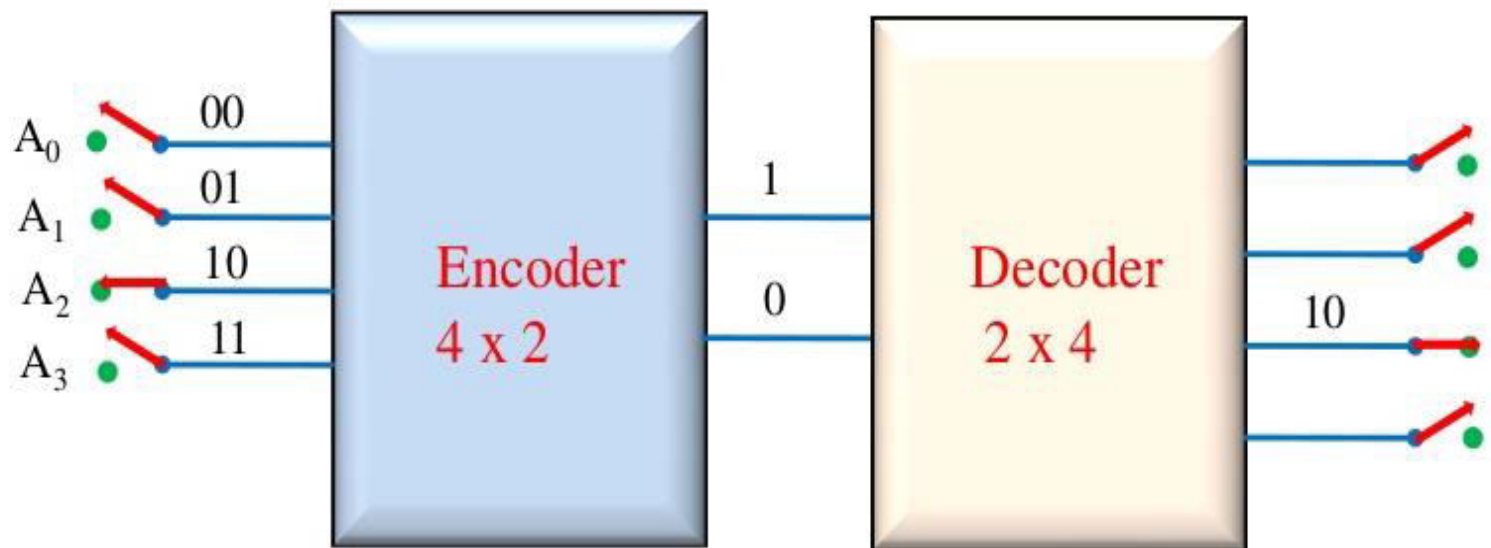
$$M=4$$

$$M=2^2$$

$$M=2^N$$

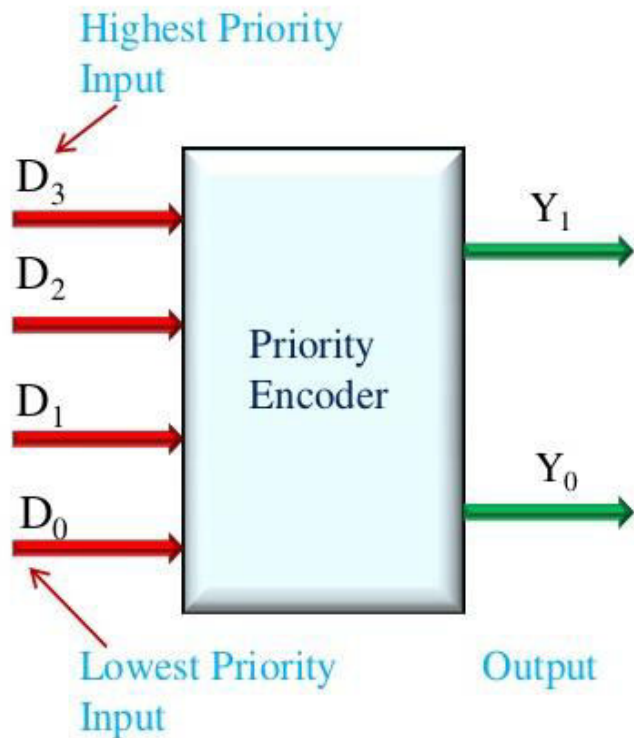
'M' is the input and

'N' is the output



PRIORITY ENCODER:

- As the name indicates, the priority is given to inputs line.
- If two or more input lines are high at the same time i.e 1 at the same time, then the input line with high priority shall be considered.
- Block diagram and Truth table of Priority Encoder are shown in fig.15



Block Diagram of Priority Encoder

TRUTH TABLE:

INPUTS				OUTPUTS		V
D_3	D_2	D_1	D_0	Y_1	Y_0	
0	0	0	0	x	x	0
0	0	0	1	0	0	1
0	0	1	x	0	1	1
0	1	x	x	1	0	1
1	x	x	x	1	1	1

Fig.15

- There are four inputs D_0, D_1, D_2, D_3 and two outputs Y_1 and Y_2 .
- D_3 has highest priority and D_0 is at lowest priority.
- If $D_3=1$ irrespective of other inputs then output $Y_1 Y_0=11$.
- D_3 is at highest priority so other inputs are considered as don't care.

K-map for Y_1 and Y_0

$D_3 D_2 \backslash D_1 D_0$		$D_1 D_0$			
		00	01	11	10
00		X	0	0	0
01		1	1	1	1
11		1	1	1	1
10		1	1	1	1

$$Y_1 = D_2 + D_3$$

$D_3 D_2 \backslash D_1 D_0$		$D_1 D_0$			
		00	01	11	10
00		X	0	1	1
01		0	0	0	0
11		1	1	1	1
10		1	1	1	1

$$Y_0 = D_3 + \overline{D_2} D_1$$

Fig. 16

LOGIC DIAGRAM OF PRIORITY ENCODER:

$$Y_1 = D_2 + D_3$$

$$Y_0 = D_3 + \overline{D_2} D_1$$

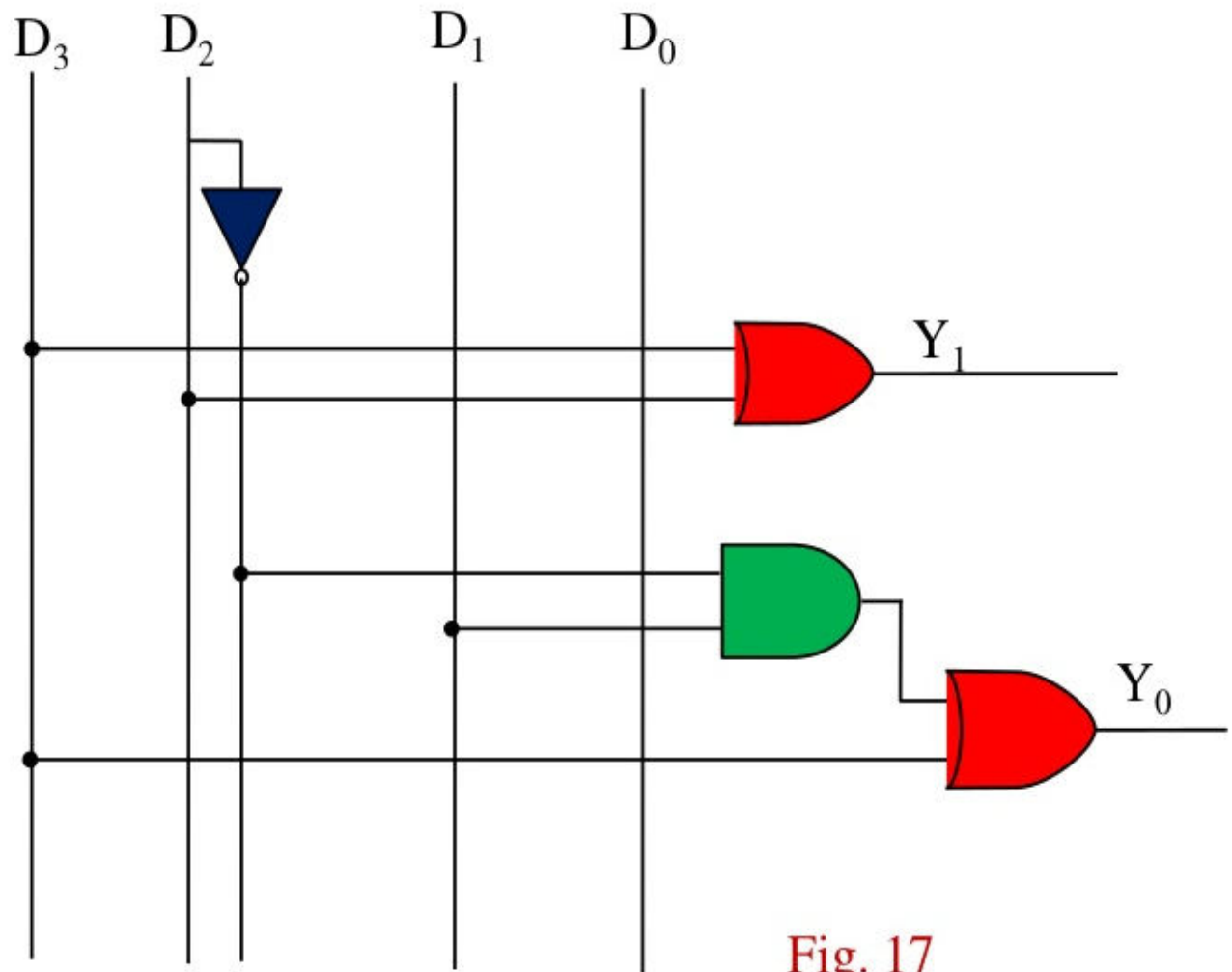
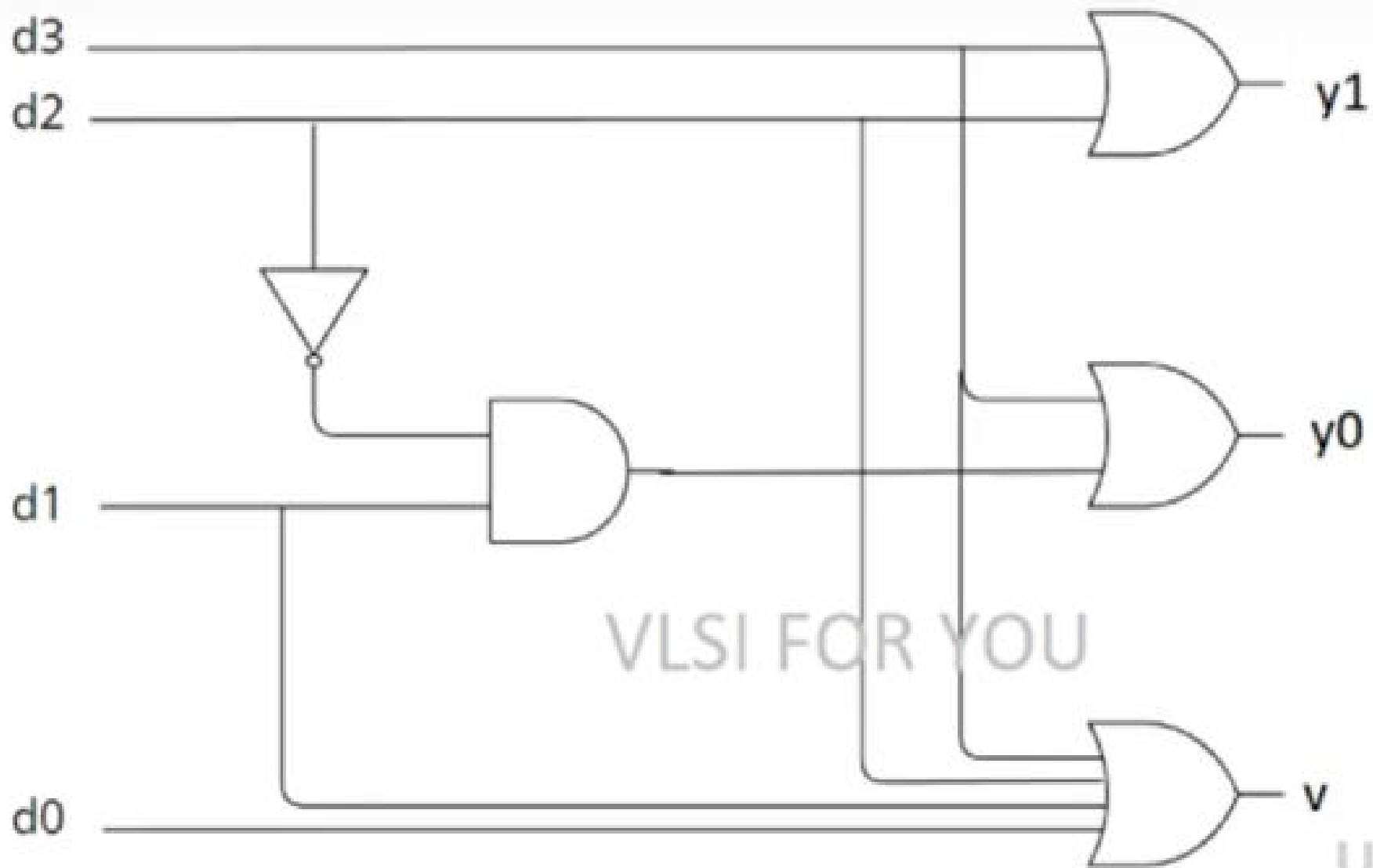


Fig. 17



Data Flow Modeling

```
module priority_encoder (input [3:0]d,  
                        output [1:0]y);
```

```
    assign y[0] =d[3] | ((~d[2]) & d[1]);
```

```
    assign y[1] =d[3] | d[2];
```

```
endmodule
```

```
module test;
```

```
    reg [3:0]d;
```

```
    wire [1:0]y;
```

```
    priority_encoder dut(.*);
```

```
    initial begin
```

```
        #2 d = 4'b0001;
```

```
        #2 d = 4'b0011;
```

```
        #2 d = 4'b0101;
```

```
        #2 d = 4'b1111;
```

```
        #20 $finish;
```

```
    end
```

```
    always @(d)
```

```
        $display( "time =%0t INPUT VALUES: d=%b OUTPUT VALUES: y=%b ",$time,d,y);
```

```
endmodule
```

VLSI FOR YOU

DECIMAL TO BCD ENCODER:

- It has ten inputs corresponding to ten decimal digits (from 0 to 9) and four outputs (A,B,C,D) representing the BCD.
- The block diagram is shown in fig.18 and Truth table in fig.19

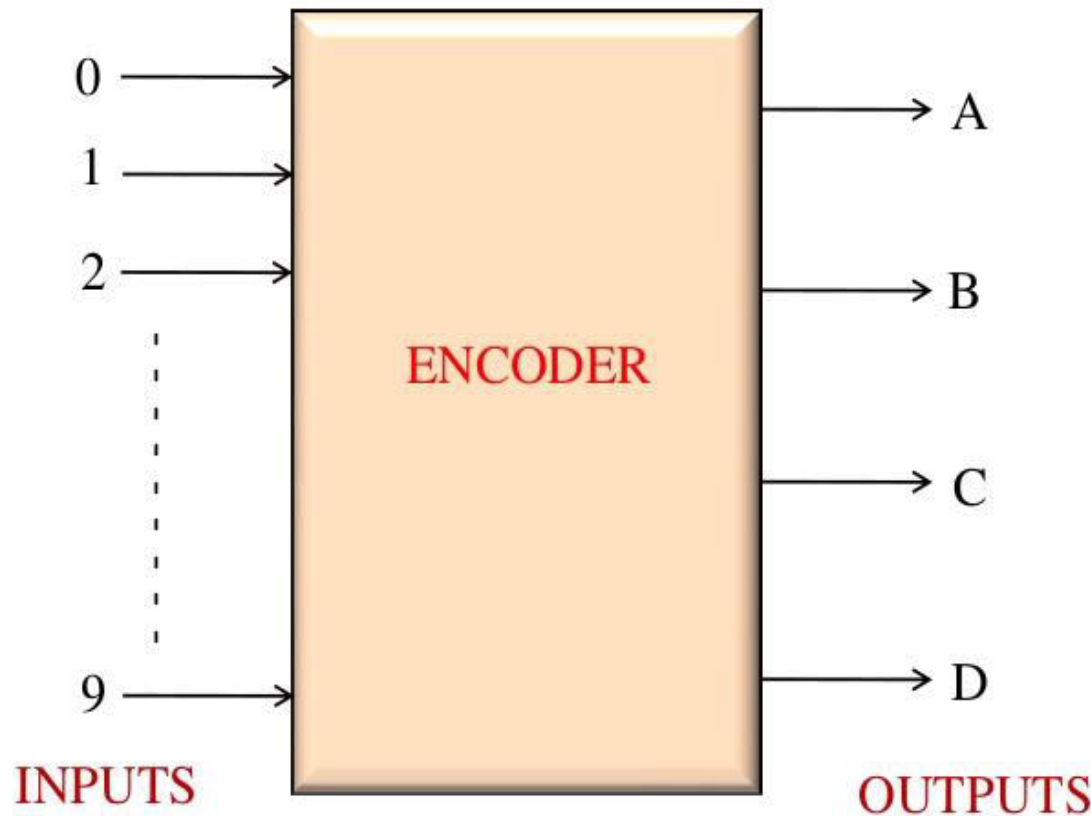


Fig. 18

Truth table:

INPUTS										BCD OUTPUTS			
0	1	2	3	4	5	6	7	8	9	A	B	C	D
1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	0	0	0	1	0	0
0	0	0	0	0	1	0	0	0	0	0	1	0	1
0	0	0	0	0	0	1	0	0	0	0	1	1	0
0	0	0	0	0	0	0	1	0	0	0	1	1	1
0	0	0	0	0	0	0	0	1	0	1	0	0	0
0	0	0	0	0	0	0	0	0	1	1	0	0	1

Fig. 10

- From Truth Table it is clear that the output A is HIGH when input is 8 OR 9 is HIGH

Therefore $A=8+9$

- The output B is HIGH when 4 OR 5 OR 6 OR 7 is HIGH

Therefore $B=4+5+6+7$

- The output C is HIGH when 2 OR 3 OR 6 OR 7 is HIGH

Therefore $C=2+3+6+7$

- Similarly $D=1+3+5+7+9$

Logic Diagram is shown in fig.20

DECIMAL TO BCD ENCODER

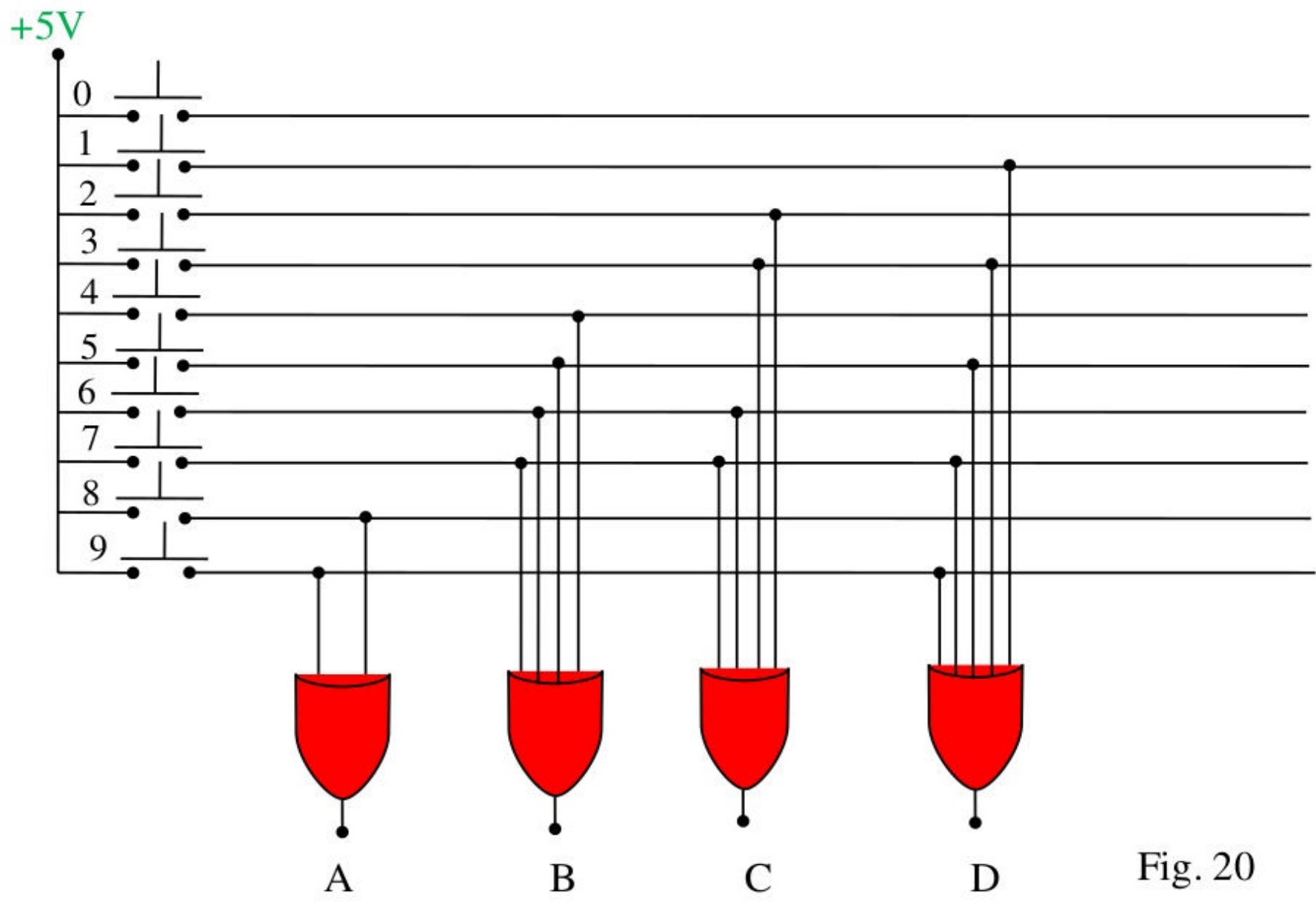


Fig. 20

OCTAL TO BINARY ENCODER:

- Block Diagram of Octal to Binary Encoder is shown in Fig. 21
- It has eight inputs and three outputs.
- Only one input has one value at any given time.
- Each input corresponds to each octal digit and output generates corresponding Binary Code.

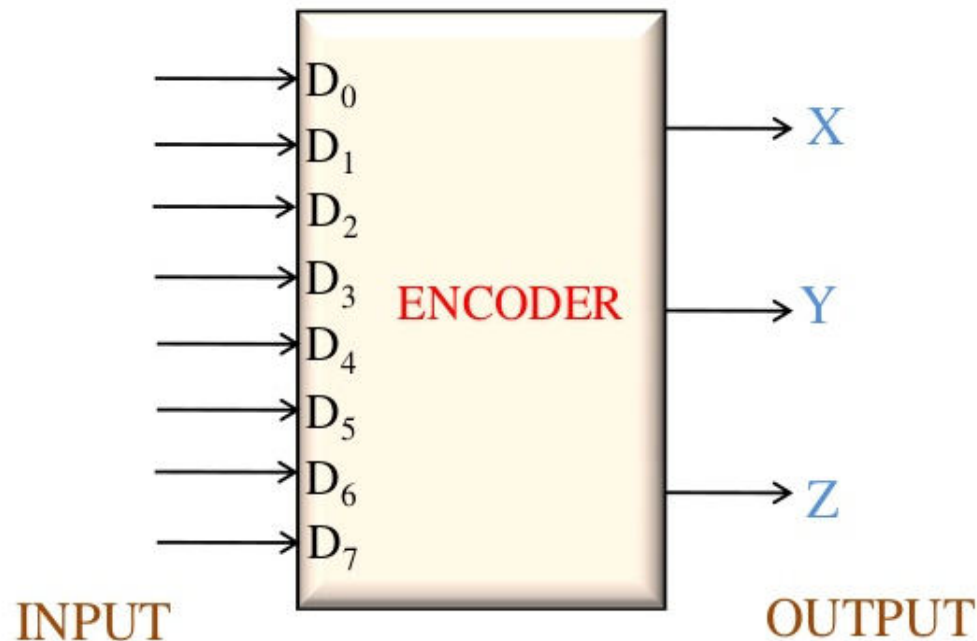


Fig. 21

TRUTH TABLE:

INPUT								OUTPUT		
D ₀	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇	X	Y	Z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

Fig. 22

From Truth table:

$$X = D_4 + D_5 + D_6 + D_7$$

$$Y = D_2 + D_3 + D_6 + D_7$$

$$Z = D_1 + D_3 + D_5 + D_7$$

- It is assumed that only one input is HIGH at any given time. If two outputs are HIGH then undefined output will be produced. For example D_3 and D_6 are HIGH, then output of Encoder will be 111. This output neither equivalent code corresponding to D_3 nor to D_6 .
- To overcome this problem, priorities should be assigned to each input.
- From the truth table it is clear that the output X becomes 1 if any of the digit D_4 or D_5 or D_6 or D_7 is 1.
- D_0 is considered as don't care because it is not shown in expression.
- If inputs are zero then output will be zero. Similarly if D_0 is one, the output will be zero.
-

$$X = D_4 + D_5 + D_6 + D_7$$

$$Y = D_2 + D_3 + D_6 + D_7$$

$$Z = D_1 + D_3 + D_5 + D_7$$

LOGIC DIAGRAM:

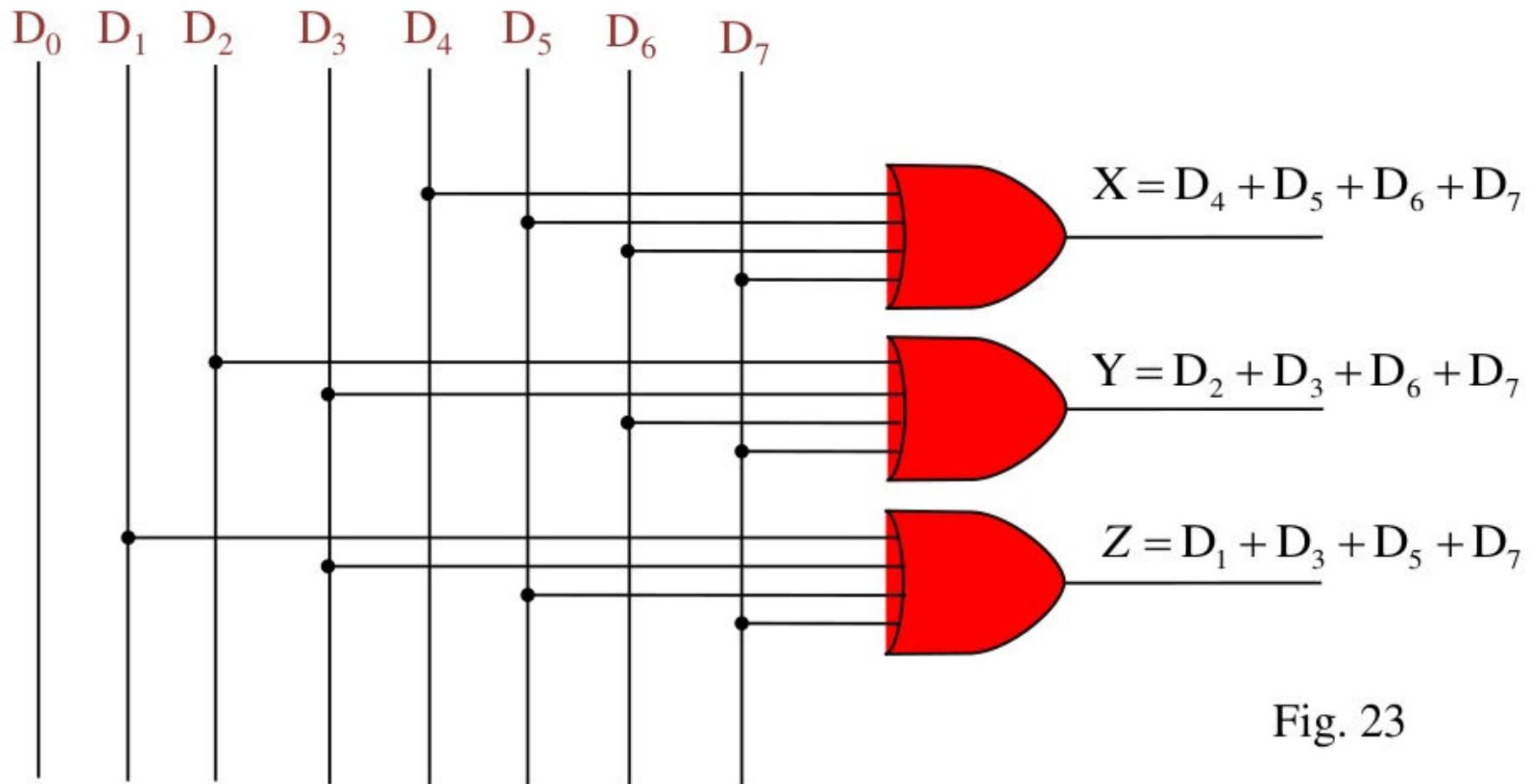


Fig. 23